

Dictionary Learning as Unsupervised Steering-Vector Generation: Ontologizers vs. Sparse Autoencoders on SONAR Sentence Embeddings

Keira Wiechecki¹ and Jade Zaslavsky¹

¹Wakaru Labs , kaw504@nyu.edu

October 4, 2026— working notes, results preliminary

Abstract

The Ontologizer is a multihead, multilayer dictionary-learning architecture whose design goal is *unsupervised steering-vector generation*: a stack of competitive classifier “heads” reconstructing an embedding space should discover the space’s natural contrastive clusters, MoEfyng the representation into small, nameable classification tasks whose within-head entry swaps are steering moves. We evaluate two forms of it — the softmax (soft-code) Ontologizer and a straight-through (hard-code) arm whose forward pass *is* the argmax — against field-standard top- k sparse autoencoders (SAEs) trained on identical cached SONAR multilingual sentence embeddings under an identical whitened reconstruction objective. The battery covers a reconstruction-vs-capacity Pareto, categorical-form and semantic-coherence tests of head structure, cross-model feature splitting and seed stability, downstream text fidelity through the paired M2M100 decoder, language probing, shrinkage, steering fidelity in text and embedding space, and blinded auto-interpretability; a *structural-ablation ladder* of SAE variants attributes effects to specific structure. Preliminary findings: (i) for soft codes, contrast-set structure is expensive in reconstruction — the plain-SAE frontier dominates the Ontologizer’s deviation code at every capacity; (ii) trained soft heads are contrast sets — within-head decode directions *repel* (96% of heads below a random-group null); (iii) categorical form exists precisely when trained in as hard selection; (iv) per-latent lenses — k -sparse probing, per-feature detection — mis-measure classifier codes, and detection F1 is largely a measure of code density; (v) a single-layer grouped-softmax SAE reproduces the reference Ontologizer’s structure and text fidelity. (vi) The straight-through arm makes the discrete code the model: its argmax and soft rows agree to four decimals where the softmax model’s differ by nine orders of magnitude, and at 1900 index bits and no continuous coefficients it reaches FVU_w 0.154, against 0.258–0.274 for converged SAEs at matched index bits that also transmit 160 coefficients — after an initialization fix worth a third of its error. (vii) At matched bits, parameters and code utilization the residual cascade is worth $1.79\times$ over a flat layer, and the measured source is the pairing between each stage’s input and the sample’s own earlier error; the same cascade makes single-head interventions leak into $\sim 60\%$ of other heads. (viii) Two seeds reach identical error while, on SONAR and on GPT-2 residuals alike, no head past the first layer — partition, contribution, subspace or Gram — reproduces above a same-shape null; first-layer heads reproduce partly, through their layer’s input gain and a few outsized atoms, and still several times worse than flat-layer heads of equal size. (ix) No head is a language variable in any arm; the strongest head is a topic/genre head. (x) Non-negativity forces a collinear dictionary at a third of the signed effective rank

for 2.4% of FVU_w . (xi) On GPT-2 small, an 800-bit hard code whose entries are signed directions in activation space recovers 97.6% of GPT-2’s loss when spliced back in, more than a $k=32$ SAE at nearly the same FVU_w (96.2%); per-entry fibers trained on the frozen code recover 99.9% without changing it.

1 Evaluation frame

The comparisons below were largely built on SAE-native terms — per-feature monosemanticity, sparse probing, reconstruction at matched sparsity — and we report them unchanged. But the Ontologizer’s goal is not a monosemantic dictionary. It is an *unsupervised generator of steering vectors*: each head is a k -way classifier over its input, trained only by reconstruction pressure, and the hope is that a series of such classifiers finds the most natural contrastive clusters in the embedding space. Interpretability value is then expected from *MoEfication*: decomposing the space into small classification tasks — one per head — each nameable in isolation, with within-head entry swaps as the steering primitive (forcing head h from its argmax entry to entry k' moves the decode along exactly the dimension head h classifies).

Under this frame the evaluation questions are: (a) what does contrast-set structure cost in reconstruction (the Pareto and ladder); (b) do trained heads actually behave as contrastive classifiers (categorical form, semantic anti-coherence); (c) are the learned tasks *natural* — nameable, stable across runs, aligned with generative factors of the data (seed reproducibility, language and topic alignment); and (d) is the steering primitive good — consistent effect, low collateral, at least matching supervised diff-of-means vectors and SAE decoder-row addition (steering fidelity). Metrics that ask whether *individual latents* are concept-aligned remain informative about the SAE side but are the wrong lens for classifier codes, and we flag them as such where they appear.

Question (c) has a prerequisite that the original battery took for granted: a head can only be named if it is the same head on every run. Section 5.7 tests that directly and finds it holds only partly, and only in the first layer, which conditions every per-head result in this document.

2 Related work

No single line of prior work covers this combination; each commitment has a nearest neighbour.

Discrete and routed dictionaries. A DictBlock head — a softmax over k entries, summed across heads — is soft product quantization [1], and the residual layer stack is soft residual VQ, the codec lineage of SoundStream [2]; that literature treats codes as compression and does not read codebooks as features. The straight-through arm (Section 5) makes the analogy literal: with an argmax forward and a softmax surrogate gradient [3] it is a residual product quantizer with non-negative atoms. Codebook Features [4] is the closest interpretability ancestor (VQ bottlenecks finetuned into a network, hidden state a sum of a few discrete codes), but its codes are read per-code, not as classifiers, and there is no intervention grammar over within-codebook swaps. Switch SAEs [5] route activations among expert SAEs purely as a compute optimization; notably their experts collapse into redundant ensembles that duplicate features — exactly the failure mode `headstruct.py/headcoh.py` instrument, and the opposite of what trained heads show here (within-head directions repel). MoEfication [6] supplies the frame’s name but

decomposes already-trained MLPs post hoc; here the partition is the training objective’s own discovery.

Structured SAEs. The field’s main structure axis is nesting by scale: Matryoshka SAEs [7] train nested dictionaries to curb feature absorption [8], and Tree SAE [9] learns explicit hierarchies. Rung 3 (`k32_p5`) imports exactly the matryoshka commitment into the ladder; the Ontologizer’s axis is instead *partition into exhaustive, mutually exclusive heads*. Both lines ask whether an architectural commitment buys interpretability that code density alone does not explain — the auto-interp results below answer “not per-latent describability,” while the description-coherence view is where the partition begins to pay.

Embedding-space substrate. SAEs over dense text embeddings became a subfield in 2025–2026: discretizing retrieval embeddings [10], data-analysis toolkits [11], human-concept alignment [12]. All are single-layer top- k SAEs, and none address SONAR, where the stakes are higher — Large Concept Models generate directly in SONAR space [13] — and where the paired M2M100 decoder supplies a semantic oracle: where embedding-inversion work must train an inverter [14], `textfid.py` and `steerfid.py` decode reconstructions and interventions through the pretrained decoder itself.

Evaluation stance. SAEBench [15] is the template for the multi-instrument battery (`langprobe.py` is deliberately SAEBench-style), and the blinded detection scoring follows the scaled auto-interp line [16]. AxBench [17] found that prompting and finetuning beat every representation-based steering method, with SAEs uncompetitive even at concept detection — independent support for judging dictionaries by steering utility rather than per-latent monosemanticity, and the niche `steerfid.py`’s effect-vs-collateral-at-matched-magnitude design targets. Song et al. [18] argue seed-to-seed feature consistency should be a first-class SAE metric; `splitting.py`’s match fractions and `partition.py`’s head matching measure precisely that. Without close analog, to our knowledge: reading grouped routing as contrastive classification with a causal intervention interface; quota-free coherence tests against size-matched nulls, including over judge-written description texts; and decoding parameter directions through the paired decoder as a test of whether features can name themselves from weights alone — a test the weights fail.

3 Methods

3.1 Architecture

We propose a novel method which encodes samples as classifications mapped to contrastive vectors. This directly constrains a model’s representations. Moreover, classifications can natively be used for steering.

The model (Figure 1) consists of l `DictEnc` layers writing into a shared residual accumulator, decoded by a single linear map. Each layer (Figure 2) is composed of a classifier layer followed by a dictionary layer. The classifier splits the input into h independent soft label vectors. The dictionary maps these labels to representative vectors. Each layer writes a weighted average of these vectors to an accumulator. At each layer, the accumulator R_i is decoded and the residual $X - \text{dec}(R_i)$ is passed to the next layer. Deep supervision is used to constrain each layer to reconstructing the input.

Because labels within a head are mutually exclusive rather than independent, each dictionary head serves as a contrastive set. An auxiliary loss term minimizing cossim_h (Eq. 6) pushes each head to represent an orthogonal subspace (Figure 3). Because the dictionary returns a linear combination of representative vectors, subspaces are convex.

3.1.1 DictEnc

Each **DictEnc** consists of a multihead bilinear classifier (**BilinearBlock**) with weights $[W, V] \in \mathbb{R}^{2 \times h \times k \times d}$ ($d+1$ after the first layer, see Eq. 14) and a multihead linear dictionary (**DictBlock**) with weights $D \in \mathbb{R}^{h \times k \times e}$. It accepts a vector $\mathbf{x} \in \mathbb{R}^d$, which may be a residual from a previous layer.

3.1.2 Classifier

A **BilinearBlock** accepts inputs $\mathbf{x} \in \mathbb{R}^d$. It returns logits $K \in \mathbb{R}^{h \times k}$ given by

$$\text{BilinearBlock}([W, V], \mathbf{x}) := \left(\sum_{j=1}^d W_{\cdot, \cdot, j} x_j \right) \odot \left(\sum_{j=1}^d V_{\cdot, \cdot, j} x_j \right) \quad (1)$$

An optional noise term ε_K may be added. Winner dropout sets $\text{argmax}_k(K_{h, \cdot})$ to $-\infty$ with probability p_{drop} , annealed from $0 \rightarrow 0.1$.

3.1.3 Labels

Soft labels P are given by $\text{softmax}_k(K/t)$ where t is a temperature hyperparameter. It is annealed geometrically from 1.0 to 0.03 over the first 50k steps, then held, giving progressively sharper classifications. The architecture provides methods for causal interventions on labels. From P we collect per-sample entropy H , in bits, so a head saturates at $\log_2 k = 5$ bits,

$$\text{entropy}_k(P)_i := - \sum_{j=1}^k P_{i,j} \log_2 P_{i,j}, \quad H := \mathbb{E}_b \mathbb{E}_h [\text{entropy}_k(P)] \quad (2)$$

between-sample similarity cossim_b , the mean absolute cosine similarity of the per-head features F over all ordered pairs of samples,

$$\text{cossim}_b(P) := \mathbb{E}_{n,q} \left[\left| \frac{\langle \mathbf{f}_n, \mathbf{f}_q \rangle}{\|\mathbf{f}_n\| \|\mathbf{f}_q\|} \right| \right], \quad \langle \mathbf{f}_n, \mathbf{f}_q \rangle = \sum_{i=1}^h \mathbf{p}_{n,i}^\top G_i \mathbf{p}_{q,i} \quad (3)$$

where $\mathbf{f}_{n,i} := F_{n,i} \in \mathbb{R}^e$ is head i 's feature vector for sample n , $\mathbf{f}_n = \bigoplus_{i=1}^h \mathbf{f}_{n,i}$ is their concatenation, and $G_i = |D_i| |D_i|^\top \in \mathbb{R}^{k \times k}$ is head i 's dictionary Gram. The implementation evaluates Equation 3 in tag space through G rather than materializing F ; the two are identical in value and gradient, but $\mathcal{O}(hk^2e + b^2hk)$ instead of $\mathcal{O}(b^2he)$.

Finally, KL_m is the divergence of the batch-mean label vector from uniform, in bits, averaged over heads.

$$\text{KL}_m(P) := \mathbb{E}_h [\log_2 k - \text{entropy}_k(\mathbb{E}_b[P])] \quad (4)$$

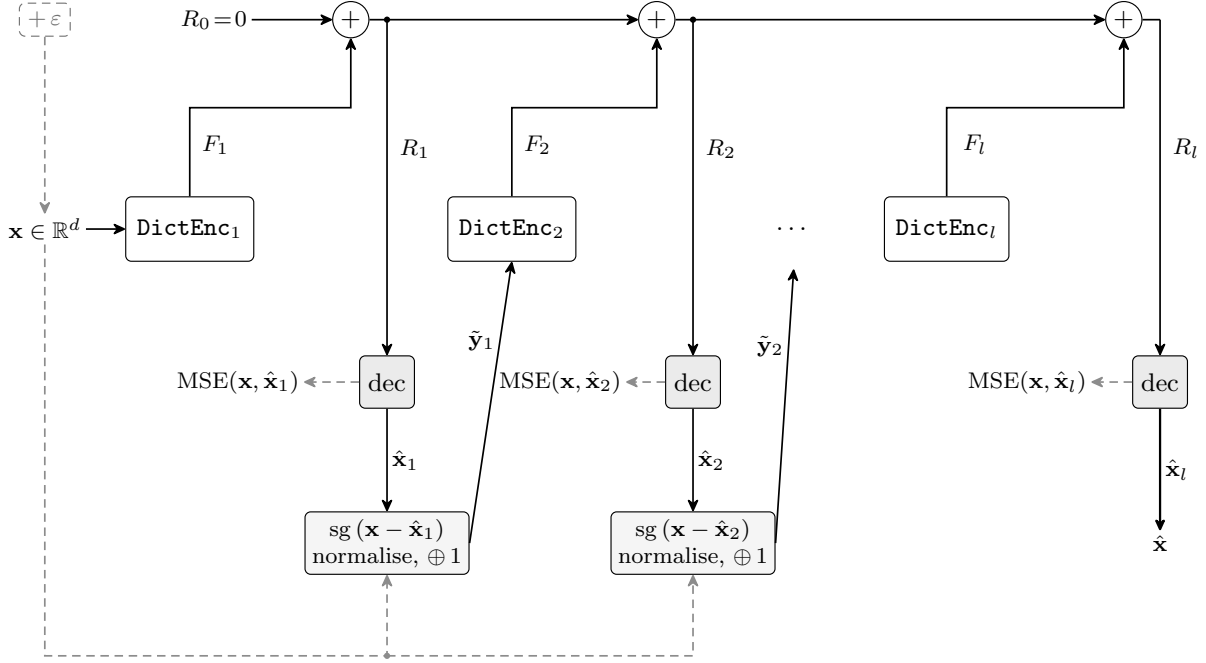


Figure 1: **Ontologizer architecture.** An Ontologizer consists of l stacked DictEnc layers. Each returns a feature vector $F_i \in \mathbb{R}^e$, which is written to a residual accumulator R . For each layer i , $R_i = R_{i-1} + F_i$; a stop-gradient variant (sg on R_{i-1} , available but off in the live configuration) would prevent gradient communication between layers. dec is a linear decoder with shape $\mathbb{R}^{d \times e}$. It uses deep supervision to obtain a coarse-to-fine representation where the component encoded by previous layers is inaccessible to later layers. At each layer i , $\hat{\mathbf{x}}_i = \text{dec}(R_i)$. The same decoder is shared by all layers. The component left unexplained after layer i is given by $\mathbf{y}_i = \mathbf{x} - \hat{\mathbf{x}}_i$. The input to layer $i+1$ is given by $\tilde{\mathbf{y}}_i = \mathbf{y}_i / \text{L2}(\mathbf{y}_i) \oplus [1]$ where L2 is the unit norm. This prevents exploding gradients from repeated application of bilinear layers, as each scales its input by $\|\cdot\|^2$. A constant coordinate $[1]$ is appended, giving a dimension of $d+1$. This prevents the sign information of $\mathbf{y}_i$ from being lost by the bilinear transformation. Effectively, it adds a bias to an otherwise unbiased linear form while preserving eigendecomposition. $\text{MSE}(\mathbf{x}, \text{dec}(R_i))$ is computed separately for each layer. The input-noise hook drawn on $\mathbf{x}$ is available but disabled in the live configuration.

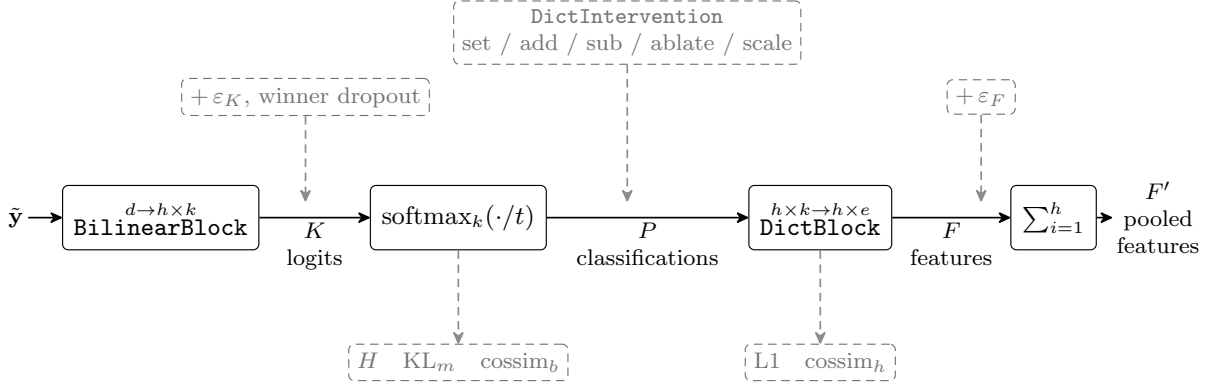


Figure 2: **DictEnc architecture.** Dashed lines above indicate perturbations to the data, including noising and causal interventions. Dashed lines below indicate statistics used as tuneable auxiliary loss functions. Each **DictEnc** consists of a multihead bilinear classifier (see §3.1.2) and a multihead linear dictionary (see §3.1.4). It accepts a vector $\mathbf{x} \in \mathbb{R}^d$, which may be a residual from a previous layer. Optional noise terms ε_K and ε_F may be added (see §3.1.6). Winner dropout sets $\text{argmax}_k(K_{h,\cdot})$ to $-\infty$ with probability p_{drop} , ramped $0 \rightarrow 0.1$ over the annealing horizon.

Soft labels P are given by $\text{softmax}_k(K/t)$ (see §3.1.3). The architecture provides methods for causal interventions on labels. From P we collect per-sample entropy H (Eq. 2), between-sample similarity cossim_b (Eq. 3), and batch mean entropy KL_m (Eq. 4).

A **DictBlock** accepts labels $P \in \mathbb{R}^{h \times k}$ and returns features $F \in \mathbb{R}^{h \times e}$ (Eq. 5). From F we collect between-head similarity cossim_h (Eq. 6) on the clean features, and the sparsity penalty L1 (Eq. 7). F is summed over the head dimension to obtain pooled features $F' \in \mathbb{R}^e$, which are added to R .

In practice H is not needed, as including a cossim_b loss also minimizes H . KL_m serves to anchor the batch mean of each label vector $\mathbf{p}_{i,\cdot}$ to a uniform distribution. This results in a uniform $\mathbf{p}_{i,\cdot}$ code corresponding to a maximally generic feature vector.

3.1.4 Dictionary

A `DictBlock` accepts labels $P \in \mathbb{R}^{h \times k}$ and returns features $F \in \mathbb{R}^{h \times e}$ given by

$$\text{DictBlock}(D, P) := \sum_{j=1}^k |D_{\cdot,j,\cdot}| P_{\cdot,j} \quad (5)$$

Because $|D_{\cdot,j,\cdot}|$ is an absolute value and $P_{hj} \in [0, 1]$, features are strictly additive. From F we collect between-head similarity cossim_h on the clean features — the mean absolute cosine similarity over all ordered pairs of heads *within* a sample —

$$\text{cossim}_h(\mathbf{F}) := \mathbb{E}_b \mathbb{E}_{i,j} \left[\left| \frac{\langle \mathbf{f}_{n,i}, \mathbf{f}_{n,j} \rangle}{\|\mathbf{f}_{n,i}\| \|\mathbf{f}_{n,j}\|} \right| \right] \quad (6)$$

and the sparsity penalty L1 after the noise ε_F is added, $\tilde{\mathbf{F}} := \mathbf{F} + \varepsilon_F$. L1 is a sum, not a mean, so it scales with b , h , and e ; its loss weight is correspondingly small.

$$\text{L1}(\tilde{\mathbf{F}}) := \sum_{n=1}^b \sum_{i=1}^h \sum_{j=1}^e |\tilde{\mathbf{F}}_{n,i,j}| \quad (7)$$

Both L1 and cossim_h tend to decrease with only MSE loss, but incorporating them into the loss speeds up convergence.

Disjoint support cossim_h is of particular interest because on nonnegative features it is not a rotation constraint but a support-partition constraint. Every $\mathbf{f}_{n,i}$ lies in the nonnegative orthant of $\mathbb{R}^e$: the dictionary contributes $|D| \geq 0$ and the labels $P \geq 0$. For nonnegative $\mathbf{u}, \mathbf{v}$ the inner product $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_j u_j v_j$ is a sum of nonnegative terms, so it vanishes if and only if every term does,

$$\langle \mathbf{u}, \mathbf{v} \rangle = 0 \iff \text{supp}(\mathbf{u}) \cap \text{supp}(\mathbf{v}) = \emptyset \quad (8)$$

Signed vectors admit no such conclusion: there orthogonality is cancellation, and two vectors with identical support can be orthogonal. This is why cossim_h ranges over $[0, 1]$ rather than $[-1, 1]$, and why the absolute value in Equation 6 is vacuous for cossim_h though not for cossim_b . It is also why the statistic is collected before ε_F is added: noised features leave the orthant and the argument with it.

The constraint binds the dictionary, not merely one sample's features. Because P is a softmax, $p_{i,a} > 0$ for every entry, and expanding Equation 5 gives

$$\langle \mathbf{f}_{n,i}, \mathbf{f}_{n,j} \rangle = \sum_{a=1}^k \sum_{b=1}^k p_{i,a} p_{j,b} \langle |\mathbf{d}_{i,a,\cdot}|, |\mathbf{d}_{j,b,\cdot}| \rangle \quad (9)$$

a strictly positive combination of all k^2 cross-head entry overlaps. A single sample on which heads i and j are orthogonal therefore forces $\langle |\mathbf{d}_{i,a,\cdot}|, |\mathbf{d}_{j,b,\cdot}| \rangle = 0$ for every pair of entries (a, b) : the two heads own disjoint blocks of coordinates as a property of D , holding for all inputs

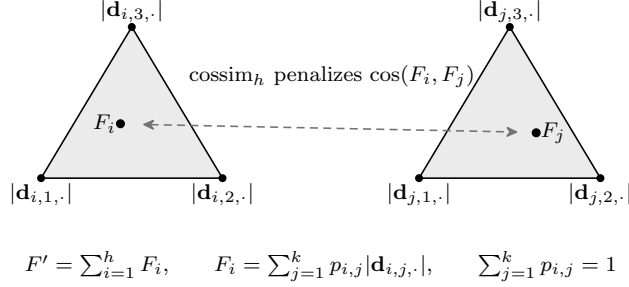


Figure 3: DictBlock **head geometry** for $h = 2$; $k = 3$. The output space of a head i forms a simplex with the rows of the matrix $D_{i,·,·}$ as the vertices. The row vectors $\mathbf{d}_{i,j,·}$ form the spanning set of the feature space. Any F_i must be a linear combination of the rows of $D_{i,·,·}$. As F becomes sparser, these vectors become more approximately orthogonal, allowing them to be treated as a basis. The label code P corresponds to soft assignments to mutually contrastive categories rather than independently firing features. Because elements of P , $p_{ij} \in [0, 1]$, outputs must be within the convex hull of $D_{i,·,·}$. Because $|D| \geq 0$ and $P \geq 0$, $F \geq 0$; therefore $\cos(F_i, F_j) \in [0, 1]$, and driving $\text{cossim}_h(F)$ to zero asks the heads to partition the coordinates of $\mathbb{R}^e$ between them (Section 3.1.4).

rather than on the data that happened to be measured. That is the premise a static reading of the dictionary needs, and it is stronger than what any signed-feature architecture can ask for. The budget is not tight either. At the live configuration $h = 32$ heads partitioning $e = 2048$ coordinates leaves 64 dimensions per head to hold $k = 32$ entries, which need not be disjoint from each other within a head; head-disjointness is most of what the width $e > d$ buys.

Two different orthogonalities are in play and only the between-head one is architectural. Disjoint support assigns each head a block of coordinates; *within* a head the k entries are contrastive alternatives spread apart inside that block (Figure 3). Section 4.4 measures the second. Neither is guaranteed at convergence: cossim_h is a penalty, and the geometry is built to make a support partition the cheap solution rather than to force one.

We add noise ε_F to F , then sum over the head dimension to obtain pooled features $F' \in \mathbb{R}^e$, which are added to R .

$$F' = \sum_{i=1}^h \tilde{F}_{i,·} \quad (10)$$

3.1.5 Decoder

Each DictEnc output F'_i is added to an accumulator R .

$$R_i = \sum_{j=1}^i F'_j \quad (11)$$

A single-layer unbiased linear decoder dec projects R_i back to the input space $\mathbb{R}^d$. Weights $W_{\text{dec}} \in \mathbb{R}^{d \times e}$ are shared for all DictEnc $_i$.

$$\hat{X}_i = W_{\text{dec}} R_i \quad (12)$$

The residual Y_i is given by $X - \hat{X}_i$, stop-gradiented so that lower layers cannot write a communication code into the residual instead of reducing it. It is normalized by the unit norm L2, with ϵ the floating-point epsilon of the working dtype.

$$Y_i = \text{sg}[X - \hat{X}_i], \quad \text{L2}(\mathbf{y}) := \sqrt{\sum_{j=1}^d y_j^2} \quad (13)$$

$$\tilde{\mathbf{y}}_i = \frac{\mathbf{y}_i}{\text{L2}(\mathbf{y}_i) + \epsilon} \oplus [1] \quad (14)$$

A constant coordinate [1] is appended, giving a dimension of $d + 1$. This prevents the sign information of $\mathbf{y}_i$ from being lost by the bilinear transformation. Effectively, it adds a bias to an otherwise unbiased linear form while preserving eigendecomposition.

3.1.6 Noise

Noise is added to X , K , and F , controlled by hyperparameters σ_X , σ_K , and σ_F . Each draws a standard normal of the same shape as its target,

$$\varepsilon \sim \mathcal{N}(0, 1) \quad (15)$$

and scales it three different ways. Classifier logits take unscaled Gaussian noise,

$$\text{noise}_{\text{normal}}(K) := K + \sigma_K \varepsilon \quad (16)$$

inputs take noise proportional to their own norm, so the scale is a fraction of signal magnitude rather than an absolute quantity,

$$\text{noise}_{\text{batchnorm}}(X) := X + \sigma_X \varepsilon \odot \frac{\text{sg}[\text{L2}(X)]}{\sqrt{d}} \quad (17)$$

and features take noise proportional to each feature’s standard deviation across the batch.

$$\text{noise}_{\text{featvar}}(F) := F + \sigma_F \varepsilon \odot \text{sg}\left[\sqrt{\text{var}_b(F) + \epsilon}\right] \quad (18)$$

Here $\text{L2}(X)$ is the per-sample norm of Equation 13, broadcast over d . Both scales are stop-gradiented: without sg the model could evade its own noise by shrinking feature variance, and the ϵ inside the root guards the 0/0 gradient where a feature’s batch variance underflows to exactly zero. Live values are $\sigma_X = 0$ (input noise off), $\sigma_K = 0.02$ annealed geometrically to $0.2 t_{\text{end}}$, and $\sigma_F = 0.1$. Because σ_K enters the softmax as σ_K/t , holding it constant under the falling temperature would grow effective exploration rather than hold it.

3.1.7 Router and fibers

Two optional branches read the same shaped input $\mathbf{u}$ as the classifier and meet it in the dictionary lookup (Figure 4). Neither is on in the live configuration.

The *router* (**scaled**) is a second bilinear map, from $\mathbf{u}$ to one scale per head, made non-negative by an absolute value (**router_signed** drops it). It multiplies its head’s whole output, so the classifier decides *what* a head says and the router *how much*; its own L1, L1_S , prices the router alone, where L1 on the output can be paid down by either factor.

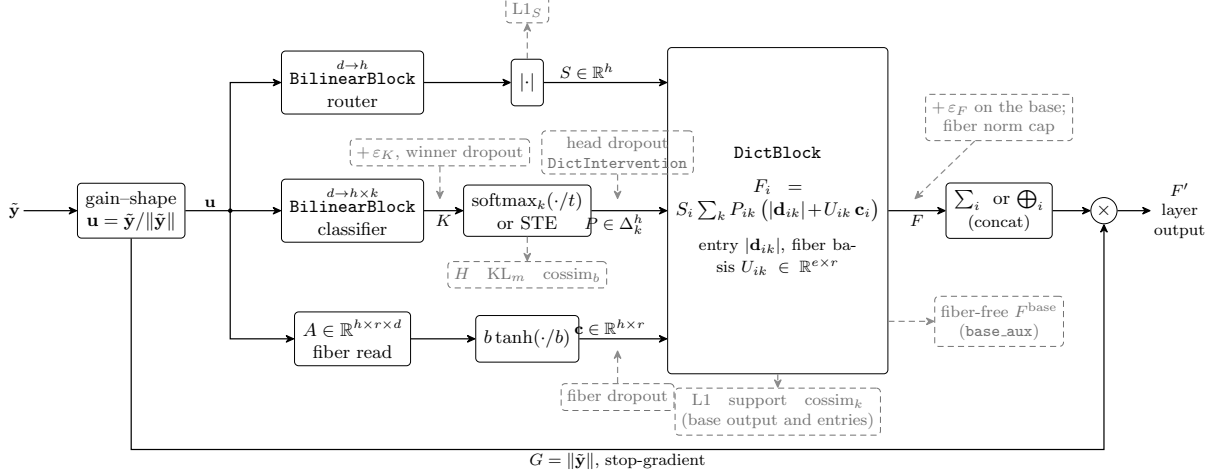


Figure 4: **DictEnc with router and fibers**. The gain–shape split hands the unit input $\mathbf{u}$ to three branches: the classifier (which entry, P), the router (how much, S , one non-negative scale per head) and the fiber read (where within the entry’s fiber, $\mathbf{c}$, r coordinates per head). They meet in the lookup (Equation 19); heads are summed, or concatenated under **ConcatDictBlock**, and the result is scaled by the measured gain G . Dashed boxes above an arrow act on it during training; dashed boxes below are statistics, which describe the fiber-free base output. That output is also returned on its own for **base_aux**.

Fibers (**fiber_rank** r) give every entry a local basis $U_{ik} \in \mathbb{R}^{e \times r}$ (in its head’s slice under **ConcatDictBlock**), and each head reads r coordinates $\mathbf{c}_i = b \tanh(A_i \mathbf{u} / b)$ from the input, so

$$F_i = S_i \sum_{j=1}^k P_{ij} (|\mathbf{d}_{ij}| + U_{ij} \mathbf{c}_i). \quad (19)$$

Under a hard code the selected entry is a discrete base point and the fiber a continuous correction within the plane its basis spans (Figure 5). Unconstrained, the correction can take over and leave the base meaningless, so the base is kept honest by fiber dropout, a norm cap ($\|U_{ij} \mathbf{c}_i\| \leq \varepsilon$ per head, or summed over heads), a base-only deep-supervision term (**base_aux**, which also decodes the fiber-free output), or training the fibers after freezing the base. A radial fiber ($r = 1$, no basis) only rescales the selected entry. Statistics describe the base output. Head dropout (**p_head_drop**) is the third addition here: in training it replaces a head’s classification by the batch mean, its origin, so heads cannot co-adapt and ablating one to its origin stays in distribution.

3.2 Training

3.2.1 Embedding model

We embed texts using SONAR[19], a multimodal and language-agnostic sentence embedding model. SONAR embeddings have several desirable properties:

- They correspond to entire texts rather than tokens.
- They are bidirectional.

inverse variance, normalized to mean one, the same whitening the evaluation applies. The implementation applies $\sqrt{\mathbf{w}}$ to X and $\text{dec}(R_i)$ before the squared error, which is equivalent. Four auxiliary statistics are active as loss terms, of six the implementation exposes (Figure 2): between-sample similarity cossim_b , computed from P through the dictionary Gram so it measures feature-space similarity of the per-head reconstructions, and which in practice also minimizes per-sample entropy H ; the batch-mean term KL_m , anchoring each head’s mean label vector to a uniform distribution so that a uniform code corresponds to a maximally generic feature; an L1 penalty on features; and between-head similarity cossim_h , which measures the extent to which heads encode nonoverlapping geometries. L1 and cossim_h tend to fall under MSE alone; including them in the loss speeds up convergence. Classification sharpness is controlled by annealing the softmax temperature geometrically from 1.0 to 0.03 over the first 50k steps, then holding it. Winner dropout and the noise terms ε_K and ε_F regularize classifications and features respectively.

3.3 Evaluation

Evaluation runs against top- k SAEs trained on identical cached SONAR embeddings under an identical whitened reconstruction objective. The battery is deliberately SAE Bench-style[15]: a reconstruction-versus-capacity Pareto; categorical-form and semantic-coherence tests of head structure; cross-model feature splitting and seed stability; downstream text fidelity through the paired M2M100 decoder; sparse language probing; shrinkage decomposition; steering fidelity; and blinded auto-interpretability scoring. A structural-ablation ladder of SAE variants, each adding one Ontologizer-style commitment at a time, attributes effects to specific structure. Ax Bench[17] found that simple supervised baselines beat SAEs at steering; the steering comparison here is built to face that result.

3.4 Data and objective

Embedding space and data. All SONAR models are trained on a fixed cache of $N=4,000,000$ SONAR sentence embeddings ($d=1024$, unit-norm) computed from an interleaved 86-language mC4 sample, with a language label sidecar. The stream order is deterministic, so raw text is recoverable by row index (verified against the language sidecar per row). The cloud is unit-normed but not centred: $\|\mathbb{E}[x]\| = 0.564$, so it is a spherical cap and 0.318 of every squared norm is one shared direction. It is dense without being isotropic — effective dimension 108 of 1024 (participation ratio 268 on the whitened eval tail), yet 943 principal components for 99% of variance — and samples average a pairwise cosine of 0.31. Sections 7.2 and 5.7 additionally use GPT-2 [20] layer-8 residual-stream activations ($d=768$, harvested by `encode_acts.py`) as a substrate where sparse dictionary learning is known to work. Section 6 uses a separate GPT-2 setup from the `headline` branch — the residual stream entering block 8 on FineWeb, unwhitened, scored also by GPT-2 loss recovered — whose numbers compare only within that section.

Evaluation. All models minimize the whitened MSE of Equation 20. We report FVU_w : whitened MSE divided by the whitened variance of a constant predictor, on the final 32,768 cache rows. This tail is excluded from SAE training and from the training of every straight-through arm. *Caveat: the softmax Ontologizers (`resid_nc` and the selection sweep) predate the holdout and trained on the full cache including the tail, so their scores are in-sample; a fresh-encoded evaluation set is future work.* Where activations are needed, Ontologizer features

are the flattened per-head classification probabilities $P \in \Delta^{k \cdot l \cdot h}$ at the run’s final temperature (feature $f = (l \cdot h + h_i)k + k_i$; fire threshold $2/k$), and SAE features are latent activations (fire threshold 0; grouped-softmax runs use the above-uniform threshold $2/(m/G)$, matching the tag convention). Scoring a softmax run at the default temperature 1.0 rather than its annealed 0.03 puts a healthy checkpoint at $FVU_w \approx 0.91$, which is indistinguishable from a broken one; every number here uses the run’s own temperature.

3.5 Models and baselines

Softmax Ontologizers. The reference Ontologizer (`resid_nc`, step 372,600) is the architecture of Section 3.1: $l=5$ residual layers, $h=32$ heads, $k=32$ entries per head, bilinear classifiers with residual normalization and a constant input coordinate, deep supervision, and a shared linear decoder from an $e=2048$ dictionary space; ≈ 23.1 M parameters and $l \cdot h \cdot k = 5,120$ dictionary entries. Layers 0–3 trained healthily (prefix FVU_w $0.129 \rightarrow 0.0051$); layer 4 froze 22/32 heads late in temperature annealing and is mildly harmful (Section 7.3 explains why). A later *selection sweep* holds that shape fixed and varies only the `DictBlock` selection rule — `top1`, `top2`, `top4`, `top8`, `top16`, `softmax` — each with and without the mean-entropy bonus raised to $s_{KL_m} = 3 \times 10^{-5}$ (`_shm`), at 369,500 steps.

The straight-through arm. `train_ste.py` imports `sonar.py`’s configuration and overrides only `select="ste"`: the forward pass selects the argmax one-hot, and the backward pass uses $\text{softmax}(K/T)$ as its surrogate. Its shape, calibration and initialization are described with its results in Section 5.

SAEs. SAEs follow Gao et al. [21]: pre-activation $\text{relu}((x - b_{dec})W_{enc} + b_{enc})$ with top- k selection, unit-norm decoder rows, tied initialization, aux- k dead-latent revival, trained with Adam at 4×10^{-4} for ~ 144 k steps at batch 4096 (converged; convergence is optimizer-step-limited, not epoch-limited, at this batch size).

The structural-ablation ladder. To attribute differences to specific architectural commitments rather than to whole architectures, we train SAE variants that each add one Ontologizer-style constraint (all $m=5120$, same data and objective):

rung	run	added structure	analogue
1	<code>k32</code>	none (top- $k=32$)	baseline
2a	<code>g160top1</code>	160 groups, hard winner per group	heads (hard)
2b	<code>g160softmax</code>	160 groups, softmax per group	heads (soft)
3	<code>k32_p5</code>	5 nested prefix losses (matryoshka)	deepsup layers
—	<code>k32_b1</code>	bilinear encoder, $[x - b_{dec}; 1]$ factors	bilinear classifier

Controls at width $m=11,264$ (parameter-matched to the Ontologizer): `k32`, `k160` (coefficient-matched to one deviation per head), and `k5120` (coefficient-matched to the full soft code; the top- k constraint never binds — measured $L_0 \approx 1520$ — so it is effectively a dense ReLU autoencoder). A seed replica `k32_s43` supports stability analysis. The bilinear encoder’s latents admit closed-form “eigenfeature” analysis: each latent’s symmetric form $\text{sym}(w_1 w_2^\top)$ is rank 2, with top- $|\lambda|$ eigenvector $w_1 / \|w_1\| \pm w_2 / \|w_2\|$ (Pearce et al. [22]). Rung 2b (`g160softmax`) is the load-bearing

control under the steering frame: it isolates the “series of softmax classifiers” idea from the residual stack, bilinear classifiers, and shared dictionary.

3.6 Evaluation protocols

Each protocol is a standalone script sharing the eval split and activation conventions; the root-level ones are unit-tested against planted-structure synthetic data.

Capacity Pareto (pareto.py). Capacity is continuous coefficients transmitted per sample, plus index bits. Ontologizer code points are *top- m deviation truncations*: per head, keep the m entries with largest $|p - \text{origin}|$, pin the rest to the origin distribution, renormalize. The origin is the corpus-mean classification $\mathbb{E}[p]$ — measured, not uniform: the model output is jointly *linear* in the flattened classifications (zero residual seed, linear decoder), so $\text{decode}(\mathbb{E}[p]) = \mathbb{E}[Y]$, which decodes to the corpus-mean generic text, while the uniform code lands off-manifold (cosine 0.067 to $\mathbb{E}[X]$) — despite KL_m being included in the loss precisely to make the uniform code the generic one (Section 3.1.3). Endpoints: $m=0$ (input-independent origin), $m=k$ (exact soft model), and end-to-end argmax (“hard”). An alternative head-level code mean-ablates all but the m lowest-entropy heads per layer. SAE points use measured eval L_0 . A hard code’s index bits are $l \cdot h \log_2 k$; we compare each against the reverse water-filling bound [23] on the whitened eval-tail covariance, the best distortion *any* code of that rate can reach under a Gaussian model.

Categorical form (headstruct.py). An Ontologizer head is exhaustive (always fires) and exclusive (one winner). We test whether SAE latents form such units: discovery clusters latents by *paradigmatic* affinity — similar co-fire profiles with the rest of the dictionary $\times$ anti-correlation with each other ($\text{PMI} < 1$) — under a group-size cap, on one half of the rows; scoring on the held-out half computes per-group exhaustiveness $P(\text{any fires})$, exclusivity $\mathbb{E}[\#\text{firing} \geq 1]$, and the CV of summed activation, against size-matched random partitions of the live-latent pool. Head-like structure requires exhaustiveness *above* null with exclusivity *below* null; topical co-firing clusters show the opposite signature. `--trained-groups` scores a grouped run’s own partition.

Semantic coherence vs. contrast (headcoh.py). The complement to categorical form: do a head’s entries *mean* similar things, or do they form a contrast set? Every entry has an exact decode direction in SONAR output space — for the Ontologizer, the rows of the linear map G with $Y = P_{\text{flat}}G$; for SAEs, decoder rows — and SONAR space is a semantic sentence space, so within-head direction similarity is semantic similarity of what entries decode to. Per head we report mean pairwise cosine and top-singular-value energy fraction, z -scored against size-matched random subsets of the same pool (same layer for Ontologizer heads); as a metric-hygiene control the same statistics are recomputed under the training objective’s inverse-variance inner product (`--metric whitened`). $z \gg 0$ means a head clusters semantically; $z \ll 0$ means its entries repel — the signature of a *contrastive classifier*, whose class decoders are pushed apart. A second view SONAR-encodes auto-interp description texts and tests pooled within-head similarity against a label-permutation null. `--assignment` scores discovered (post-hoc) groups.

Feature splitting and seed stability (splitting.py, partition.py, headcontrib.py). For models A (coarse) and B (fine) on shared rows, each B -latent is classified per A -parent by

two-way firing containment at threshold τ : a *child* satisfies $P(A_i | B_j) \geq \tau$, $P(B_j | A_i) < \tau$; a *match* satisfies both. Splits (≥ 2 children) are scored by union coverage of the parent’s firing set and pairwise child overlap. Decoder geometry adds absorption candidates (decoder-cosine twins that do not co-fire) and each model’s nearest-neighbour decoder-cosine distribution vs. a random baseline. For two same-architecture runs differing only in seed, the match fractions are the reproducible-feature fractions. For hard-code Ontologizers, `partition.py` matches heads across runs by the NMI of their partitions (what a head separates), and `headcontrib.py` by the cosine of their contributions (what a head writes): each head’s decoded output contribution in the whitened frame, less `decode(0)`, centered over rows — uncentered, the non-negative dictionary’s shared offset reads as agreement. Both use Hungarian matching, a row-shuffled null, and the same run against an earlier checkpoint as the positive control. `headcontrib.py` also splits each head’s contribution size into its layer’s input gain and its own decoded atoms, and with a reference seed pair (a flat arm) compares agreement at matched head size.

Text fidelity (`textfid.py`). The “loss recovered” analogue: decode x and $\text{recon}(x)$ through M2M100 (forced `eng_Latn`, both rescaled to the reference SONAR norm, so the metric sees direction error) and report chrF [24] and exact-match between the decodes, plus the decoder’s teacher-forced per-token NLL of the *original’s* decode under the original (ceiling), the reconstruction, and the corpus-mean embedding (floor — which decodes to the generic text). Under the steering frame this doubles as the collateral ceiling: a code that cannot preserve text cannot steer text with low collateral.

Language probing (`langprobe.py`, `headlang.py`, `headscript.py`). SAEBench-style k -sparse probing with the free 86-language labels: top latents per language by two-sample t -statistic on training rows; held-out one-vs-rest F1 at $k=1$ (single latent, fitted threshold), $k=4, 16$ (logistic probes), and a dense probe on the raw embedding as ceiling. Macro-F1 over languages. *Frame caveat: this measures whether a factor lives in individual latent magnitudes. A classifier head carries its factor in its assignment (partition), which single-latent probes cannot see.* `headlang.py` is the matching instrument: per head, the NMI between its partition and the label, null-corrected by a label shuffle, plus a held-out probe on the best m heads’ one-hot codes against a dense ridge ceiling. NMI here normalizes by the arithmetic mean of the two entropies, so its attainable maximum is $2 \min(H_h, H_\ell) / (H_h + H_\ell) = 0.875$ for a 5-bit head against the 6.43-bit language label. `headscript.py` repeats this for script and scores per-label tag cells and cross-head conjunctions; `headeta.py` measures each partition’s η^2 in embedding space, corrected by the $(g-1)/(n-1)$ a random g -way partition scores.

Shrinkage (`refit.py`). Gao-style refinement: freeze each sample’s support, solve the whitened least-squares coefficients (masked normal equations; silent group slots forced to zero), and compare FVU_w . Raw-refit is magnitude (shrinkage) error; refit is selection error. The Ontologizer shares the machinery via its exact linearity $Y = P_{\text{flat}}G$, with G measured by decoding one-hot constant codes; its supports are the top- m deviation entries. Points with support $\geq d$ are excluded (underdetermined). Note the raw convention: `refit.py` truncates the soft forward’s P (non-adaptive), whereas `pareto.py` truncates inside the forward, letting later layers reclassify against the truncated prefix residual.

Steering fidelity in text (steerfid.py). Unit-normalized native steering directions — SAE decoder rows, bilinear eigenfeatures, or the Ontologizer’s `withArgs` set-intervention delta — applied at matched embedding magnitudes. Effect is cycle consistency: the steered decode is re-encoded through SONAR and the target feature’s activation gain is normalized by its firing-conditioned quantiles, with the baseline taken from the *cycled* base decode so both sides share roundtrip degradation. Collateral is $1 - \text{chrF}$ against the base decode. Random unit directions give the control curve, and *net* effect is feature effect minus that control.

Steering fidelity in embedding space (steerembed.py). Drops the text round trip and asks the causal question directly: move the input along a direction, re-run the model, and record whether the intended entry is now selected (*realization*) and what fraction of the *other* heads changed their selection (*collateral*). Directions: **decode** (the decode delta of forcing the entry), **grad** (the gradient of the target logit), **margin** (the gradient of the target’s margin over the head’s current winner), **adjoint** (the bilinear classifier’s top eigenvector via `BilinearBlock.rev`), and **random**. 64 entries, 32 held-out rows each, steered only where the entry was not already selected.

Intervention composition (compose.py). Treating heads as candidate causal variables: for pairs of forced assignments $(l_1, h_1) \rightarrow k_1$, $(l_2, h_2) \rightarrow k_2$ (8 random heads per layer, one random entry each; all same-layer pairs plus 60 cross-layer pairs), compare the joint intervention’s decode delta D_{12} against the sum $D_1 + D_2$ of the single-intervention deltas on 512 held-out rows: median per-row cosine and relative error $\|D_{12} - (D_1 + D_2)\| / \|D_1 + D_2\|$, raw and whitened. Because the decode is exactly linear in the flattened classification, final-layer pairs compose exactly by construction; that stratum doubles as the measured numerical floor (the $\|R\| \approx 31 \rightarrow \|Y\| \approx 1$ decoder cancellation amplifies float32 rounding to ~ 0.02 relative error between differently-compiled variants — verified exact in float64). All other non-additivity is downstream reclassification: later layers re-reading the intervened residual.

Auto-interpretability (autointerp.py). Four stages: harvest (top-activating rows, negatives, frequency-stratified feature selection, parameter-based decode embeddings), texts (deterministic mC4 stream replay recovers text by row index, verified against the language sidecar), describe, and score. Description modes: **params**, the M2M100 decode of a parameter-derived embedding (for tags, the $\mathbb{E}[p]$ -origin code with one head forced one-hot; for SAE latents, $b_{dec} + \alpha W_{dec,j}$); **pdev**, the same decode with the pure-origin decode subtracted; **eig**, the bilinear eigenfeature decode; **acts**, an LLM summary of top-activating snippets; and **cacts**, a *contrastive* summary — the judge sees top snippets next to random corpus draws and must name what distinguishes them, blocking the degenerate corpus-level answer that plain summaries invite on noisy multilingual web text. Scoring is blinded detection over held-out positives and negatives, identical for all modes; per-feature F1, to be read against the always-match null of $F1 \approx 0.67$ on balanced items. The judge is Haiku (`claude-haiku-4-5`) via the Anthropic Messages API, rate-metered, with bounded reply length; failed judgements are never recorded (an earlier outage mode scored empty replies as $F1 = 0$). Six models: the Ontologizer, rungs k32/k32_b1/g160top1/g160softmax, and the dense control k5120.

Diagnostic scripts (experiments/ste-arm/). The straight-through and geometry results rest on further single-purpose instruments, described where used: **inputgeom.py** (geometry of the vector each classifier sees), **blendablate.py** (severing the pairing between a layer’s input and

the sample’s own earlier reconstruction), `gainablate.py` (the norm side-channel), `dictgeom.py` (row collinearity and effective rank read from the weights), `lastlayer.py` (final-layer ablations against the deep-supervision prefixes), `decodehead.py` (decoding a head’s entries as deviations from its mean), `embedgeom.py` (SONAR cloud geometry), `entryshare.py` (how evenly a layer’s entries contribute to the output, and whether usage, gain or atom norm drives the inequality), and `bigatoms.py` (how a layer’s large atoms are distributed among its heads, and whether the heads holding them reproduce).

4 Results: the softmax Ontologizer against SAEs

4.1 The price of structure: reconstruction–capacity Pareto

point	coeffs	index bits	FVU _w
onto $m=0$ ($\mathbb{E}[p]$ origin)	0	0	1.001
onto hard (argmax)	0	800	17.34
rate floor at 800 bits	0	800	0.226
sae k32 tier ($m=5120/11264$, incl. p5, s43)	32	~400	0.494–0.509
sae k32 bilinear	32	394	0.602
onto dev $m=1$	160	800	0.677
sae g160top1	160	1972	0.274
sae k160 ($m=11264$)	160	2154	0.258
onto dev $m=2$	320	1600	0.497
onto dev $m=4$	640	3200	0.289
onto dev $m=8$	1280	6400	0.094
sae k5120 ($m=11264$, $L_0 \approx 1520$)	1520	20455	0.00011
onto dev $m=16$	2560	12800	0.0082
sae g160softmax	5120	0	0.00436
onto soft ($m=32$)	5120	0	0.00538

Table 1: Reconstruction vs. code capacity for `resid_nc`, shared eval tail. The plain-SAE frontier dominates the Ontologizer’s deviation code at every measured capacity; the gap is the price paid for contrast-set structure in a soft code.

Findings. (1) At matched FVU_w ≈ 0.49 , the SAE transmits 32 coefficients where the Ontologizer needs 320 (a $10\times$ coefficient-efficiency gap), with the caveat that top- m points are post-hoc truncations of a model never trained to truncate. (2) The Ontologizer’s granularity floor (one deviation per head, 160 coefficients, FVU_w 0.677) is worse than the SAE’s 32-coefficient point — the very sparse regime is structurally closed to it. (3) The parameter- and coefficient-matched dense control (k5120) reaches FVU_w 1.1×10^{-4} with $3.4\times$ fewer active coefficients than the soft Ontologizer: unstructured reconstruction is $\sim 50\times$ better at matched budgets. (4) Head-level codes (mean-ablate all but the m lowest-entropy heads) are far weaker than deviation codes at matched coefficients (heads 8/32: 0.50 vs. dev $m=8$: 0.094): reconstruction information is spread near-evenly across all 160 heads — under the classifier reading, every head’s assignment carries signal for every input, which is what a MoEfied code should look like, and why per-sample head *confidence* does not localize reconstruction. (5) Uniform-origin truncation is strictly worse than $\mathbb{E}[p]$ -origin (dev $m=1$: 0.98 vs. 0.68): the measured mean classification, not uniform, is the code’s true origin; neutral ablations and steering baselines should pin to it. (6) The soft model’s own discrete reading is not a code at all. The argmax row is $77\times$ above the best distortion *any*

800-bit code can reach (0.226), and on the selection sweep’s `softmax_shm` run the gap is nine orders of magnitude (argmax FVU_w 8.7×10^6 against 0.0007 soft). The tags that `decode.py`’s interventions and `decode_tags.py` read are therefore not what carries a softmax model’s embedding — the motivation for Section 5. Even a perfect 800-bit code would sit near FVU_w 0.23: the live head count is too small for a discrete ontology to be possible at all.

4.2 Structure decomposition (the ladder)

rung	FVU _w	L ₀	note
k32 baseline	0.499	32	0 dead
g160top1	0.274	160	1434/5120 dead (28%)
g160softmax	0.0044	5120	beats <code>resid_nc</code> (0.0054)
k32_p5	0.509	32	deepsup tax $\sim 2\%$
k32_bl	0.602	32	1848/5120 dead (36%)

Table 2: Converged structural rungs, $m=5120$, 144.3k steps each.

The grouped-simplex code constraint alone costs $\sim 30\times$ vs. the unstructured dense control (0.0044 vs. 1.1×10^{-4}); against `resid_nc`, everything the Ontologizer adds beyond rung 2b — bilinear classifiers, five residual layers, the shared 2048-d dictionary space — adds *nothing* to reconstruction (slightly negative), at $2.2\times$ the parameters. That verdict is specific to the reference checkpoint: the selection sweep’s `softmax_shm` run, identical in shape, reaches 0.0007 on the same tail (in-sample, like `resid_nc`), six times below `g160softmax`. The stack can therefore beat its single-layer rival on reconstruction; whether it does so out of sample is open. Hard-per-group selection costs only $\sim 6\%$ over an unstructured code of equal sparsity (`g160top1` 0.274 vs. `k160` 0.258), which also shows the Ontologizer’s catastrophic argmax code (FVU_w 17.3; one-hot selection inflates output norm $3.7\times$ because winner entries carry $\sim 4\times$ the soft-mixture norm) is a training artifact, not intrinsic to discrete head codes — Section 5 confirms this from the Ontologizer side. Nested prefix losses are nearly free. The bilinear encoder costs $\sim 20\%$ FVU_w and kills a third of the dictionary. Under the steering frame, the ladder’s implication is pointed: *the classifier-forming ingredient is grouped softmax itself*; whether the full stack earns its extra cost must be decided by task naturalness and steering quality, not reconstruction.

4.3 Categorical form

At 1M rows with split-half scoring and 20 size-matched nulls: `g160top1`’s trained partition is *perfectly* head-like (exhaustiveness 1.0000 vs. null 0.64; exclusivity 1.0000 vs. 1.51; sum-CV 0.25 vs. 1.03). Discovery on every unstructured run (`m11264_k32`, `m5120_k32`, `k32_p5`, `m11264_k160`) finds the *opposite* signature — exhaustiveness below null (z from -30 to -50), exclusivity above ($z +13$ to $+31$): spontaneous SAE group structure consists of co-firing topical clusters, not categorical slots, even when the affinity explicitly selects for anti-correlated same-context pairs. And `g160softmax` never fires distinctly at all: no coefficient ever exceeds $2/32$ on any test row — soft grouped codes converge to dense near-uniform mixtures, exactly as the Ontologizer’s soft code does ($\mathbb{E}[p]$ sits 0.06–0.16 bits from uniform in healthy layers; matching soft reconstruction requires ~ 16 of 32 deviations per head). **Categorical structure exists precisely when trained in as hard selection, never emerges from softmax competition under a reconstruction objective, and costs $\sim 60\times$ in FVU_w (0.274 vs. 0.0044) at the SAE’s**

160 heads. For steering this means the *soft* architectures’ classifier reading must be cashed out in deviations around $\mathbb{E}[p]$, not in discrete winners — or the selection must be made hard during training, which Section 5 does at a far smaller cost once the head count is raised.

4.4 Semantic coherence: heads are contrast sets

grouping	groups	mean $z_{\cos}$	frac $z>2$	frac $z<-2$
g160top1 (trained, hard)	160	+161.3	1.00	0.00
k32 discovered (m11264)	359	+1.7	0.36	0.02
onto heads (trained, soft)	160	-17.6	0.04	0.96
g160softmax (trained, soft)	160	-14.7	0.00	1.00

Table 3: Within-head decode-direction coherence vs. size-matched same-pool nulls (`headcoh.py`, $n_{\text{null}}=500$).

This is the sharpest dissociation in the battery, and the one the steering goal predicts. Hard-trained groups are hugely *coherent* (every **g160top1** group far above null: winner-take-all carves the data so each group’s entries tile one decoder cluster — fine-grained substitutions within a region). Post-hoc co-fire clusters are mildly coherent (topical). But both soft classifier architectures are *anti-coherent*: in 96–100% of heads, the entries’ decode directions are *more mutually dissimilar than random same-pool sets*. Softmax competition pushes a head’s class decoders apart — within-head entries are well-separated alternatives, i.e. heads function as *contrastive classifiers*, and within-head swaps are large, clean moves along the head’s own dimension. Notably the signature is shared by the Ontologizer and single-layer **g160softmax** with near-identical strength. The choice of inner product is not doing the work: recomputed under the whitened metric the training objective actually optimizes, the repulsion *strengthens* for the Ontologizer (mean $z_{\cos}$ -24.4 vs. -17.6 raw; frac $z<-2$ unchanged at 0.96) and is essentially unchanged for **g160softmax** (-14.7) and **g160top1** (+179.3 vs. +161.3).

The hard-code Ontologizer agrees from the weights side. Its within-head row cosine, decoded into output space, is 0.023 at layer 0 and -0.015 to -0.020 at layers 1–4: a head’s entries are essentially orthogonal, and slightly repelling, in the space that matters. The same statistic read in the e -dimensional dictionary space instead reads 0.12–0.29; that reading is dominated by the decoder’s null space and should not be used (Section 7.1). Of the two orthogonalities in Section 3.1.4, then, the within-head one is measured and holds in both code types.

Two `resid_nc` outlier heads (layer-2 h25, $z+346$; layer-3 h28, $z+280$; the next candidates sit in the partially frozen layer 4) cluster instead of contrasting — their sampled entries also decode to generic text, so we suspect partial head collapse (entries never differentiated), not semantic coherence; targeted inspection pending. The description-text view was uninformative on plain summaries ($\sim 77\%$ generic corpus descriptions for tags, 49% for k32 latents, inflating the permutation null: pooled $z = -0.25$); re-run on the contrastive **cacts** descriptions it acquires signal. Within-head description similarity clears the head-label permutation null for **g160top1** ($z=+2.42$, 62 within-head pairs), is marginal for Ontologizer heads ($z=+1.92$, 155 pairs), and is absent for **g160softmax** ($z=+0.79$, 217 pairs). Read against the geometry this is the pattern “alternative values of one shared dimension” predicts: Ontologizer entries’ decode *directions* repel while their described *contents* weakly cohere. It is also the first instrument in the battery to separate the Ontologizer from its single-layer soft rival, whose head contents show no description

coherence at all — suggestive rather than significant at this coverage (one global z per model over the campaign’s sampled features).

4.5 Feature reliability

Seed stability (**k32** vs. **k32_s43**; identical architecture and data, FVU_w 0.4994 vs. 0.4967): matched-feature fractions are **0.31 / 0.24 / 0.15 / 0.03** at $\tau = 0.3/0.5/0.7/0.9$. Roughly three quarters of individual features are not reproducible across initializations at moderate containment. Cross-width splitting (**k32** $\rightarrow$ **m11264_k32**): 190 of 5114 live parents split at $\tau=0.7$ (median 3 children; union coverage 0.37; child overlap 0.42), with 441 absorption candidates and a heavy nearest-neighbour decoder-cosine tail (p50 0.234 vs. random 0.119): splitting is real but is not the dominant organization; topical co-firing is. For a steering generator the unit that must reproduce is the head’s partition rather than the individual feature vector; Section 5.7 measures that on the hard-code Ontologizer and finds it reproduces far *worse* than SAE features do.

4.6 Text-space fidelity (collateral ceiling)

model	chrF	exact	NLL (ceil. ≈ 1.56 – 1.63 , floor ≈ 3.8)
corpus-mean floor	≈ 0.20	0	3.60–3.89
k32 tier (incl. p5, m11264)	0.23	0.00	3.00–3.08
k32_bl	0.21	0.00	3.19
k160 / g160top1	0.28–0.29	0.00	2.34–2.37
onto soft	0.538	0.090	1.658 (gap 0.026)
g160softmax	0.592	0.119	1.563 (gap 0.007)
k5120	0.856	0.592	1.587 (gap 0.000)

Table 4: Downstream text fidelity, 512 tail rows, forced English decodes.

The FVU–text mapping is sharply nonlinear: $FVU_w \approx 0.5$ decodes to same-topic-different-facts text barely above the generic floor, while the Ontologizer’s soft code and **g160softmax** are near the likelihood ceiling — almost all decodable content survives. Across the eleven Ontologizer runs with both numbers (the selection sweep and the straight-through arms; Table 6), “loss recovered” is a monotone function of FVU_w (rank correlation essentially -1), so text fidelity is a check that the whitened objective transports to text, not an independent axis — and it does. This matters doubly under the steering frame: a steering primitive that routes through a code which cannot preserve text (the entire $k \leq 160$ sparse tier) starts from ~ 0.75 collateral before any intervention, whereas the dense-mixture classifier codes start near zero.

4.7 Concept (language) localization

Language remains distributed in every model (best single-latent macro-F1 0.119 vs. dense ceiling 0.676; script-distinctive languages such as Chinese reach ~ 0.98 from one latent, same-script families smear), and localization *anticorrelates* with reconstruction across the family. The Ontologizer’s tag probabilities are decisively the worst localizers ($F1@1 = 0.022$). Read on SAE terms this is an indictment; read on classifier terms it is expected: a head encodes a factor in its *assignment*, not in any single tag’s magnitude, and partition-level analysis of **resid_nc** found layer-1 heads whose partitions carry script/family mutual information, one of which ((L1,h8,k17)) was

model	F1@1	F1@4	F1@16	dense
k160	0.119	0.393	0.473	0.676
k32 ($m=5120$)	0.089	0.347	0.465	0.676
m11264_k32	0.078	0.364	0.471	0.676
k32_bl	0.084	0.228	0.368	0.676
g160softmax	0.068	0.162	0.328	0.676
g160top1	0.066	0.200	0.302	0.676
k32_p5	0.061	0.322	0.454	0.676
k5120	0.064	0.130	0.237	0.676
onto (tags)	0.022	0.021	0.117	0.677

Table 5: Macro one-vs-rest F1 over 86 languages, held-out tail.

causally verified to force Burmese output. The partition-level instrument (`headlang.py`) was later run on a config-matched softmax reference and on the hard-code arms (Section 5.8): the softmax reference’s best head reaches a null-corrected NMI of only 0.043, a sixth of the hard code’s. Language factors are carried by a *few* heads’ partitions, weakly, and most heads classify something else.

4.8 Shrinkage decomposition

Top- k SAEs have small magnitude error (share of raw FVU_w removed by support-frozen least squares: 2.2–2.4% for k32 variants, 8.5% for k160, 0.6% bilinear, 1.9% p5); **g160top1** is essentially perfectly calibrated (0.3%) — its reconstruction deficit is pure selection. The Ontologizer’s truncated deviation codes are magnitude-heavy: 16% at $m=1$, 33% at $m=2$, i.e. the pin-and-renormalize convention mis-scales what it keeps (supports of $\geq d$ coefficients are excluded as underdetermined). Separately, the gap between non-adaptive truncation (freeze soft P , truncate, decode linearly) and in-forward truncation (later layers reclassify against the truncated prefix residual) is large — 0.268 vs. 0.094 at $m=8$: downstream layer compensation is worth $\sim 3\times$ under truncation. This was the first quantified reconstruction credit for the residual stack (Section 5.4 measures a larger one), and it is steering-relevant: the stack actively re-routes around code edits, which cuts both ways — robustness to small interventions, and, as Section 5.5 shows, widespread collateral.

4.9 Steering fidelity

In text. On the SAE side only a smoke-scale run exists (k32, 3 features): cycle-consistency effect rises with magnitude ($1.2 \rightarrow 1.8$ over magnitudes $0.5 \rightarrow 1.0$ relative to the unit input norm; hit rate $0.67 \rightarrow 0.89$), while collateral at these magnitudes (≈ 0.75) is near the random-direction control. Across the Ontologizer selection sweep and the straight-through arm, at 64 features per run:

Steerability does not track reconstruction. The best steerer is **top8** at FVU_w 0.037, the most faithful model (**softmax_shm**) is among the weakest, and every **_shm** variant steers about three times worse than its twin (**top2** 0.474 vs. **top2_shm** 0.141; **top8** 0.580 vs. 0.193). The straight-through arm’s effect is positive but nearly flat in magnitude, and its hit rate (0.068) barely clears its own random control (0.062). Two cautions on this instrument. Sixteen features is too few to quote: at 16 features the straight-through arm’s net effect was negative at every

run	FVU _w	chrF	loss rec.	net @0.25	@0.5	@1.0	hit @1.0	rand. hit
top8	0.0370	—	0.940	0.131	0.303	0.580	0.713	0.277
top2	0.3516	0.248	0.532	0.125	0.289	0.474	0.564	0.217
top16	0.0052	—	—	0.065	0.119	0.243	0.313	0.139
top8_shm	0.0287	0.416	0.956	0.072	0.096	0.193	0.291	0.142
softmax_shm	0.0007	0.694	0.998	0.032	0.072	0.146	0.150	0.064
top2_shm	0.1911	0.303	0.738	0.022	0.070	0.141	0.154	0.041
top4_shm	0.0761	0.360	0.897	—	—	—	—	—
ste_h76	0.2293	0.282	0.702	0.072	0.096	0.107	0.068	0.062
top1	0.7190	0.180	0.123	0.011	0.026	0.048	0.069	0.020

Table 6: Text fidelity and text-cycle steering (`steerfid.py`, `--n-features 64`) across selection rules. Net effect is feature effect minus the random-direction control. A further run (`topk4_rev`, FVU_w 0.164) reads net 0.520 at 1.0.

magnitude and at 64 it is positive throughout. And the hit rate is not comparable across hard and soft codes: under a hard code an activation is exactly 0 or 1, so a “hit” demands the argmax land exactly on the target entry, where a soft code need only cross a fractional threshold.

The text cycle is too weak to support conclusions. Measured on an *unsteered* round trip (512 held-out rows), the cycled embedding retains barely more than the shared corpus direction: $\cos(x, \text{cycle}(x))$ is 0.378 for `ste_h76` and 0.380 for softmax, against 0.314 between random pairs, and a head’s argmax survives the round trip on 12.1% and 10.5% of rows against 3.1% chance (34.8% at layer 0, 4.6% by layer 4 for the hard code). A perfect intervention could therefore be observed at most $\sim 12\%$ of the time — the same order as the hit rates the instrument reports. Reconstruction into embedding space is fine; generation is where the signal goes.

In embedding space. `steerembed.py` removes the round trip:

direction	ste_h76 @0.25 / 1.0	softmax @0.25 / 1.0
decode	0.289 / 0.591	0.824 / 0.569
grad	0.236 / 0.113	0.453 / 0.477
adjoint, before sign fix	0.007 / 0.016	0.224 / 0.329
adjoint, oriented	0.008 / 0.021	0.427 / 0.501
adjoint, oriented per sample	0.026 / 0.026	0.446 / 0.512
random	0.023 / 0.030	0.020 / 0.037

Table 7: Realization rate (target entry newly selected), 64 entries $\times$ 32 held-out rows, chance 0.031. Collateral agrees within 0.01 across directions at a given strength, so rows compare at equal budget.

Steering works, and the text cycle was hiding it. The decode direction realizes the target entry on 82% of rows for the softmax model at strength 0.25, where the text cycle put every model within noise of random. **The hard code is the less steerable of the two:** at matched collateral (~ 0.58) softmax realizes 0.82 against the hard code’s 0.28, and does so at the lowest strength while the hard code needs the largest — a hard argmax has margins a small nudge cannot cross. Sweeping strength on the hard code, its decode direction saturates at ~ 0.66

(a third of interventions are unrealizable at any strength), and it overtakes the soft model only past collateral 0.85, where most other heads have moved too.

BilinearBlock.rev had a **sign bug**: **eigh** fixes eigenvectors only up to sign, so **rev** returned a direction whose sign came from LAPACK. **nlinear.orient** now fixes it by the eigenvalue’s sign with a leading-coordinate tie-break. The fix roughly doubles eigenvector steering on the soft model ($0.224 \rightarrow 0.427$ at 0.25), within noise of orienting per sample (0.446). That is more than theory allows: moving x along v changes $x^\top Bx$ by $2\epsilon\lambda(x \cdot v)$, a property of the sample that a weight-only convention should not be able to supply. It can here because SONAR is anisotropic — $x \cdot v$ is dominated by $\mathbb{E}[x] \cdot v$, which has a consistent sign — whereas on isotropic Gaussian inputs (the unit test) a weight-only direction raises the logit on about half the samples. The hard code is unmoved by either orientation. The depth decomposition of these numbers, which reverses the aggregate ranking of **decode** and **grad**, is in Section 5.5.

Still not run: SAE decoder-row addition and supervised diff-of-means vectors in the embedding-space instrument, and per-head effect consistency across contexts. The existence proof from **resid_nc** stands: forcing (L1,h8,k17) reliably yields Burmese decodes of otherwise preserved content.

4.10 Description specificity (auto-interp)

Under the identical plain summarization prompt, 51% of **k32** latents receive a specific description (20% naming a language or script) versus 23% of Ontologizer tags (8%): individual SAE latents are about twice as describable as individual tags, consistent with tags not being standalone features. Blinded detection over the full campaign agrees, and sharpens the reading:

model	density	freq	params	eig	acts	cacts	<i>n</i>
onto (resid_nc)	100%	10.80%	0.020	—	0.473	0.392	225
k32	0.6%	0.61%	0.158	—	0.645	0.603	250
k32_bl	0.6%	2.62%	0.061	0.021	0.578	0.470	75
g160top1	3.1%	5.08%	0.039	—	0.646	0.594	150
g160softmax	100%	0.00%	0.025	—	0.449	0.364	256
k5120 (dense ctrl)	45.5%	17.97%	0.001	—	0.470	0.332	256

Table 8: Mean per-feature detection F1 by description mode (always-match null ≈ 0.67 ; a fully generic description that matches every item scores the null, so generic descriptions *inflate* toward it). Density is nominal code density; freq is the firing rate at the $2/k$ threshold. **k32.bl**’s pool is only 75 live features. No null control was recorded for this campaign.

Three readings. (1) *Per-latent describability tracks code density, not architecture*. Activation-based descriptions rank **k32** $\approx$ **g160top1** (0.65) > **k32.bl** (0.58) > onto $\approx$ **k5120** (0.47) > **g160softmax** (0.45). Pearson(log nominal density, **acts**) is -0.894 across the six runs, so density alone carries $\sim 80\%$ of the variance. Fitting **cacts** $\sim a + b \log(\text{firing rate})$ on the four SAE runs with nonzero firing rate (slope -0.069 , $r = -0.766$) and holding the Ontologizer out predicts 0.421 against an actual 0.392, a residual of -1.8 sem: the Ontologizer sits on the SAE trend. **g160top1** is the real outlier, $+0.120$ above it — the Ontologizer’s own categorical code in SAE form, and unusually describable for its density. The fit is thin ($n=4$, weak r ; **g160softmax** is excluded because its thresholded firing rate is exactly 0, and the $2/k$ threshold is non-monotone in true density), but the lesson is durable: **per-latent detection F1 cannot compare architectures at unmatched density, and every soft Ontologizer is dense by construction**. (2) *The*

contrastive mode is the honest number. `cacts` scores below `acts` everywhere, but the drop is smallest for the sparse rungs (-0.04 to -0.05) and largest for the dense control (-0.14): part of every plain-summary score is generic-match inflation, and the contrastive prompt removes most of it. (3) *Parameter-space decodes are not descriptions.* `params` is ≤ 0.06 everywhere except plain `k32` (0.158), and the bilinear eigenfeature decode fails identically (`eig` 0.021). Part of that is a decoding error: an entry’s cosine to its head mean is 0.849, so a raw decode returns one shared template sentence with details shuffled, and anything resting on `decode_tags.py` reads the template rather than the feature. Subtracting the origin decode (`pdev`) makes the decodes legible but not detectable — on one head, `pdev` scores 0.052 against a 0.014 null, `params` 0.031 against 0.000, and `cacts` 0.479 against 0.273. A decoded sentence is an *instance* of what an entry writes, and detection needs a criterion. Naming requires activation evidence.

5 A hard-code arm: straight-through selection

The softmax model’s discrete reading is not its code (Section 4.1, finding (6)), so its tags are not what interventions act on. Under `select="ste"` the forward pass *is* the argmax and no such gap can open: at every checkpoint scored, the hard and soft-forward rows of `pareto.py` agree to four decimals.

5.1 Capacity, and why the head count rises

A hard code transmits exactly $l \cdot h \log_2 k$ index bits. Against the reverse water-filling bound on the whitened tail:

FVU _w target	bits	heads at $k=32, l=5$
0.226	800	160 (the softmax architecture)
0.05	1,898	380 (this arm, $h=76$)
0.0023	4,167	833

Heads are the cheap axis: bits and parameters are both linear in h , and heads run in parallel where layers are sequential. The arm is 5×76 , $k=32$, 1900 bits, 51.93M parameters.

5.2 Calibration

A naive port (live configuration, `select="ste"`) trains but the classifier learns nothing. The root cause is one property: **the argmax forward is scale-invariant, so nothing pushes the logits up**; they sit at their initialization scale (within-head standard deviation 0.0015) for the whole run. Three settings follow from it. *Logit noise must be relative*: $\sigma_K=0.02$ is thirteen times the logit spread, so the argmax is chosen by noise, while $\sigma_K=0$ removes the only exploration and the codebook collapses; `--noise-k batchnorm` (Equation 17) scales noise by the logit RMS. *The surrogate temperature must track the logit scale*: at the live floor $T=0.03$, spread/ T is 0.05 and the surrogate is flat, so what improves is the dictionary adapting to a fixed random partition. *s_{KL_m} must be rescaled*: 10^{-6} is calibrated against a whitened MSE of $\sim 3 \times 10^{-6}$, and a hard code trains at an MSE orders of magnitude larger, where the term is inert and heads collapse.

T	spread/ T	max p	effective entries/head	raw FVU
0.01	0.15	0.043	23.4	178
0.0015	0.78	0.135	26.4	193
0.0005	3.38	0.681	14.0	59.2
0.00015	16.84	0.974	3.8	7.3

(4000 steps, lr 10^{-5} , `--noise-k batchnorm --sd-k 0.3`; one seed per cell, which ranks configurations but does not predict final quality.) Sharpening the surrogate is what makes the classifier train at all and improves reconstruction 25-fold, but it trades directly against head usage, which s_{KL_m} exists to hold. At `sonar.py`'s learning rate 5×10^{-5} every sharp-surrogate arm collapsed abruptly after ~ 1700 steps; 10^{-5} is stable. The shipped arm is $T=1.5 \times 10^{-4}$, lr 10^{-5} , `--noise-k batchnorm --sd-k 0.3`, $s_{KL_m}=10^{-4}$, trained 24 epochs (369,500 steps). The temperature is hand-matched to a logit scale measured once; the principled version normalizes logits per head or controls T against a spread/ T setpoint.

5.3 Initialization: a third of the error

The dictionary is `abs()`'d, so a layer's h rows sum coherently and its output scales with h ; the per-layer gain compounds that, so the initial residual grows as roughly $(ch)^l$. The shipped arm starts at 1.84×10^8 where it should start near 1. `--dict-init-scale 0.1` rescales each dictionary so a layer contributes about a tenth of its input, which divides h out and starts the cascade as a near-identity:

arm	shape	s_{KL_m}	realized bits	FVU _w	params
<code>ste_h76</code> (shipped init)	5×76	10^{-4}	1714 (90.2%)	0.2293	51.93M
<code>ste_h76_init01</code>	5×76	10^{-4}	1895 (99.7%)	0.1535	51.93M
<code>ste_h76_i01_s43</code> (seed 43)	5×76	10^{-4}	1895 (99.7%)	0.1535	51.93M
<code>ste_k2_h380</code>	5×380 , $k=2$	5×10^{-4}	1887 (99.3%)	0.1844	17.67M
<code>ste_l1_h380_i01_hm1e4</code>	1×380	10^{-4}	1898 (99.9%)	0.2750	51.93M
<code>ste_l1_h380_i01</code>	1×380	5×10^{-4}	1898 (99.9%)	0.2870	51.93M
<code>ste_l1_h380_i01_s43</code>	1×380	5×10^{-4}	1898 (99.9%)	0.2871	51.93M

Table 9: Straight-through arms at the fixed initialization (except the first row), held-out tail. Realized bits are $h(l \log_2 k - KL_m)$, computable from `loss.csv`.

A third off the held-out error and the code essentially full, from one scalar — the largest single effect in the straight-through work. It subsumes two earlier corrections: the utilization gap that the entropy-pressure rescale and input whitening were each partly recovering *was* the initialization. **The steering, language, gain and blend measurements below were made on the shipped initialization** and have not been remeasured; the seed-reproducibility and entry-share measurements of Section 5.7 use the fixed one.

A purely discrete code beats a sparse linear one. Scored on one frontier of converged SAEs (the structural ladder plus an L1 sweep at the same ~ 144 k-step schedule; `pareto.py` into `data/out/sonar/pareto_unified`):

code	coeffs	index bits	FVU _w
straight-through , 5×76	0	1900	0.154
SAE g160top1	160	1972	0.274
SAE m11264_k160	160	2154	0.258
SAE ReLU+L1 3×10^{-5}	519	6390	0.220
SAE ReLU+L1 10^{-5}	1438	17723	0.048

At matched index bits ($\sim 2,000$) the discrete code is 40% below SAEs that additionally transmit 160 continuous coefficients, and it is 30% below an L1 SAE spending $3.4\times$ the bits plus 519 coefficients. Truncation rows (**dev** m) are *worse* than the hard row on this arm (0.259 at $m=1$; 0.343 on the shipped initialization): pinning entries toward the origin adds mass the model was never trained to decode, so for a hard model the honest capacity axis is $l \cdot h \log_2 k$ itself. Against each code’s own floor, depth is the less rate-efficient use of bits:

code	floor	achieved	ratio
380 bits (one layer’s prefix)	0.4208	0.6427	$1.53\times$
1900 bits, 5×76	0.0499	0.1535	$3.08\times$
1900 bits, 5×380 , $k=2$	0.0499	0.1844	$3.70\times$

Bits are not the whole capacity. At identical bits, 380 binary heads reach 0.1844 against 76 thirty-two-way heads at 0.1535: bits go as $h \log_2 k$ but the codebook goes as $h \cdot k$ (3800 atoms against 12,160). The binary arm gets within 17% on $2.9\times$ fewer parameters, with features that are one bit each — the form that is actually nameable.

Training dynamics. On the shipped arm, reconstruction converges early (MSE flat from step 237k) while within-head row collinearity keeps falling ($0.812 \rightarrow 0.686$, against ~ 0.64 for random non-negative rows) and head-usage imbalance KL_m falls from a 14.8-bit peak near step 5k to 2.71. The s_{KL_m} rescale is what turns the usage peak around.

5.4 Depth at fixed capacity

Five layers of 76 heads and one layer of 380 have the same nominal capacity (1900 bits) and exactly the same parameter count (51.93M; both the dictionary *hke* and the classifier *hkdn* depend on l and h only through $l \cdot h$). At the fixed initialization both fill 99.9% of their code, and **depth is worth $1.79\times$ the reconstruction error** (0.1535 vs. 0.2750). Neither seed nor undertraining explains it: seed replicas agree to four digits, and the flat arm’s training MSE moves -0.55% over its last quarter while a sibling run $2.5\times$ longer ends no better held out.

The comparison went through two wrong versions first, and the eliminations are informative. On the shipped initialization the flat arm read 0.5822 at 484 realized bits (25.5%), a $2.5\times$ gap. Correcting the per-head entropy pressure — KL_m averages over a layer’s heads and the loss sums over layers, so per-head pressure scales as s_{KL_m}/h per layer and the flat arm had trained at a fifth of the stack’s — took it to 1199 bits and 0.4307. A frozen ZCA whitener (`make_zca.py`) took the classifier’s input from effective dimension 107 to 901 and the code to 1900 of 1900 bits, every head at 4.999 of 5 bits, more than the stack itself realized — and still reconstructed $1.7\times$ worse (0.3842). Two survivors from that line: **usage is not what depth buys** (bits are not the objective either: dropping s_{KL_m} 100 $\times$ costs the stack 176 bits and *improves* its error to 0.2095), and **a learned input transform collapses where a supplied one does not** — a

jointly trained **Linear** encoder ends with every head on exactly two entries at FVU_w 0.6853, worse than no encoder.

The input each classifier sees. `inputgeom.py`, on the exact vector handed to each classifier (shipped arm):

layer	mean direction	mean pairwise cos	effective dim / 1024
0	0.5638	0.3146	107.5
1	0.0932	0.0087	611.3
2	0.1522	0.0233	596.3
3	0.1983	0.0396	548.8
4	0.2348	0.0553	507.1

Layer 0 (and every flat arm) classifies the raw cap; layers 1–4 classify residuals that span over half the space. On the shipped arm, layer 0 was the only layer that did not fill its code (10.6 live entries, 216 bits, against ~ 375 for each later layer) — a within-model control on everything except the input.

What the cascade contributes: pairing. `blendablate.py` blends the subtracted term $\text{decode}(R_i)$ in layer $i+1$ ’s input toward a reference with the same marginals and none of the pairing — another row’s reconstruction (shuffle), the mean, or Gaussian noise rescaled to the shuffle’s per-sample magnitude — without retraining:

blend	shuffle	mean	noise	eff. dim L1 (shuffle)	(noise)
0.000	0.2292	0.2292	0.2292	499.1	499.1
0.125	0.2460	0.2374	0.2426	476.0	504.7
0.250	0.3084	0.2652	0.2838	394.2	518.3
0.500	0.6910	0.4100	0.4665	193.4	539.6
0.750	1.8154	0.7497	0.8502	103.0	527.6
1.000	4.6613	1.4302	1.5992	68.5	496.2

The pairing is load-bearing well below full severance — a quarter blend costs 35% of FVU_w — and another row’s reconstruction does more damage than matched-magnitude noise at every level, so the damage is specific rather than generic perturbation sensitivity. The sweep cannot separate pairing from conditioning: effective dimension falls in lockstep with the error, because a residual is small and nearly isotropic *because* it is this sample’s own error. That resolves the whitened arm rather than conflicting with it: whitening is conditioning without pairing, and it bought full utilization and $\sim 7\%$ of FVU_w . **The cascade’s value is the pairing; the conditioning is its shadow.** A flat layer is a product quantizer — no head can see another’s error — while the residual stack is a residual quantizer, each stage coding what earlier stages missed. (`forward="labels"`, which would test this by removing the residual entirely, has not been run at the fixed initialization. Forwarding the code *alongside* the residual, `forward="resid_labels"`, was a clear negative in a three-epoch pilot: held-out FVU_w 0.826 against 0.249, after leading early by $50\times$ — the signature of a parameter advantage without an information advantage, since the residual is by construction what the code failed to explain.)

The code is not the whole channel. Each layer splits its input into a unit direction (all the classifier sees) and a norm that multiplies the layer’s output, so one real number per layer per sample reaches the decoder outside the bottleneck. Pinning each gain to its held-out mean (`gainablate.py`) costs the stack +2.1% in total (layer 0’s gain is identically 1, since SONAR is unit-norm), against a 68% gap: the channel is real and is not the explanation. It does matter as a confound for the whitened flat arm, whose whitener creates a varying layer-0 gain worth +4.5% — about 40% of whitening’s apparent benefit.

The decoder is dense, and not because of the offset. Magnitude-pruning the trained decoder’s smallest half takes FVU_w into the thousands, and its weight distribution looks like a Gaussian matrix’s. The non-negative dictionary makes R ’s constant part dwarf its varying one ($\|\mathbb{E}R\|/\mathbb{E}\|R - \mathbb{E}R\| = 11.6$), and the bias-free decoder does attenuate that constant to the embedding mean (norm 0.581 against 0.564). But a decoder bias (`--biased-dec`) is worth only −0.62% of training MSE at matched step (the run died at 23% of its schedule, so its held-out 0.2464 is not comparable to the converged arm’s): pinning $W\mathbb{E}R$ is 1024 linear constraints on a 1024×2048 matrix, 0.05% of its freedom. The decoder is dense because a full-rank map carrying a ~ 900 -dimensional signal is dense.

5.5 Entanglement: the cost of the cascade

A single-head embedding-space intervention in the flat model realizes its target on 97% of rows while disturbing 9% of the other heads; in the stack it realizes 28% while disturbing 59%. Restricting the stack to layer 0, so the target is the same distance from the input, the gap does not close (realized / collateral, strength 0.25):

direction	stack, layer 0	flat
decode	0.068 / 0.616	0.966 / 0.086
grad	0.688 / 0.563	0.969 / 0.063
margin	0.513 / 0.570	0.979 / 0.069
random	0.005 / 0.584	0.064 / 0.093

The collateral is not a property of the intervention: a random push of the same norm disturbs 58% of the stack’s heads and 9% of the flat model’s. The residual cascade turns *any* input perturbation into widespread downstream reclassification; the flat model has nothing downstream, so its heads are independently addressable by construction. This is the steering-side counterpart of `compose.py`’s finding that non-additivity decays with the number of layers below an intervention (Section 8).

Splitting the stack’s steering by target layer (strength 1.0, 12 entries per layer) reverses the aggregate ranking of directions:

layer	decode	grad	margin	adjoint	random
0	0.216	0.583	0.547	0.237	0.034
1	0.464	0.089	0.367	0.000	0.023
2	0.742	0.130	0.216	0.000	0.055
3	0.966	0.130	0.091	0.000	0.034
4	0.362	0.047	0.036	0.005	0.047

Where the input reaches the classifier directly (layer 0), the classifier’s own geometry (**grad**, **margin**) beats **decode** three- to tenfold. With depth it fails: a direction computed against layer i ’s classifier input is only valid there, and the perturbation arrives after earlier layers have re-encoded it. The **decode** direction moves the input toward what the model itself emits with the tag active, which is self-consistent down the stack. Layer 4 is hard for everything: its residual is nearly exhausted. The practical reading is to steer layer 0 by gradient and the middle layers by decode.

Head-health levers and steering. Three levers were tried on the shipped stack. *Mean-entropy pressure is not what costs steerability*: a twin trained at $s_{\text{KL}_m}=10^{-6}$ has raw text-cycle effect 0.039 against 0.029, both an order of magnitude below **top2/top8** at the same weight, so hard selection, not the bonus, is the cost (removing the bonus does cost head usage: imbalance 5.00 bits against 2.71). *Head-independence pressure (HSIC/CKA) was chasing estimator bias* (Section 5.6). *A row-collinearity setpoint is the one lever that bought steerability*: holding within-head row cosine at 0.5 (against an uncontrolled 0.774) improves embedding-space realization in every direction at lower collateral — **decode** +15%, **grad** +17%, **margin** +52%, oriented **adjoint** +122%, collateral 0.591 $\rightarrow$ 0.552 — with held-out FVU_w slightly better (0.2157 vs. 0.2293). The controller holds its setpoint as a thermostat: the dual multiplier is projected to zero while the constraint is satisfied. Its mechanism is unexplained: the statistic is computed in the e -dimensional dictionary space, where 74–78% of each atom’s energy is in the decoder’s null space (Section 7.1), and decoded the entries were already orthogonal. That the classifier-geometry directions gain most suggests an effect on the classifier rather than the dictionary, but nothing establishes it. The setpoint does not transport to the flat arm, whose steering is already saturated (0.992 vs. 0.966).

5.6 Head independence: a measurement artifact

A head-independence penalty (`--s-hsic-heads`, pairwise head CKA [25]) appeared to find substantial dependence. It was measuring bias:

layer	0	1	2	3	4	mean
unbiased	0.0041	0.0005	0.0005	0.0004	0.0002	0.0011
biased	0.0315	0.1062	0.1018	0.0963	0.0976	0.0867

The biased estimator reads chance co-occurrence between independent categorical heads as dependence, with a floor that grows as the batch shrinks (0.109 at $b=256$, 0.029 at 1024, against an unbiased 0.0001 at every size). At the training batch the floor exceeded any real dependence, so the penalty had nothing to descend but the bias. `hsic.pairwise_head_cka` now defaults to the unbiased estimator [26], which reads 0.06 at 0.5 planted factor correlation and 0.41 at 0.8. Conditioning on language or script explains none of the residue. **This architecture produces near-independent heads with no pressure to do so**; trained out under the corrected estimator the penalty changes nothing and costs localization (best-head language NMI 0.199 against 0.269). Two lessons generalize: before trusting a statistic as a training signal, measure it on a null it should score zero on; and loss-value share is not gradient share — at $s=10^{-4}$ the penalty was 12% of the loss value, while the measured MSE-to-CKA gradient ratio (1250 at the classifier, 221 at the dictionary) put equal pull at 8×10^{-4} .

5.7 Seed reproducibility: the function is determined, the heads are not

Two runs differing only in seed, both at the fixed initialization, reach **identical** held-out error — 0.1535 and 0.1535 for the stack, 0.2870 and 0.2871 for the flat arm (Table 9). Nothing that identifies an individual head agrees:

what is compared	stack	flat	null
head partitions (mean matched NMI)	0.0121	0.0177	0.0048
head contributions (centered cosine)	0.0074	—	0.0015
per-layer subspace, rank 8	= data’s	—	see text
per-head tag Grams	1.05–1.11× chance	—	within-layer pairing

Partitions. The best-matched pair of heads reaches NMI 0.42, but the mean is 0.012 against a 0.005 null, and `splitting.py`’s tag containment reads 0.000. Neither is a metric artifact: comparing one run against its own checkpoint 19,500 steps earlier, the same partition measure scores 0.38 overall and 0.87 on layer 0. *Contributions.* Two heads could carve differently and add the same thing; they do not (centered cosine 0.0074 vs. 0.0015 null; uncentered it reads 0.205, which is the non-negative dictionary’s shared offset). `headcontrib.py`, which fixes the measurement (decoded, whitened, centered contributions, Hungarian-matched), reads 0.019 against a 0.0017 null on the same pair at 4,096 rows; the earlier one-off figure’s exact method was not recorded, and the conclusion is unchanged under either. *Subspaces.* Top-8 eigenspaces of per-layer contributions agree 25–33× above chance — but each model agrees with the *cache*’s own principal directions by the same amount (e.g. layer 1: 123.7× between seeds, 101.3× and 101.2× to the data), so the agreement is the data’s and nothing is left for the architecture. At layer 4 each seed tracks the data $\sim 23\times$ above chance while agreeing with the other only $9\times$: they pick *different* data-aligned directions. *Grams.* Per-layer row collinearity agrees to four digits (0.2880 / 0.2891 at layer 0), but the Gram spectrum is nearly one-parameter — the top eigenvalue is exactly $1 + (k - 1)$ rowcos, carries 82% of the spectrum’s norm, and is forced by non-negativity. Compared entrywise in canonical order, Grams agree at chance across seeds ($1.05\text{--}1.11\times$) and *equally* at chance against the same run 19,500 steps earlier ($1.07\text{--}1.15\times$): a head’s Gram is close to a fixed property of the configuration rather than something learned. Decoding atoms to output space first changes nothing ($1.18\times$ at layer 0 to $1.03\times$ at layer 4).

What reproduces is aggregate or layer-level; what identifies an individual head does not. Feature-level claims about individual heads have no foundation in this architecture as it currently trains, and every per-head result in this document — the topic head below, the Burmese head, the per-head steering tables — inherits that. Three of the four measures reversed under a control before they said anything (containment, uncentered contributions, subspaces), and an earlier Gram comparison read $35\times$ against a whole-model random pairing, entirely the layer signal. The rule: **in this architecture non-negativity and the layer-dependent gain put structure into every measure’s floor, so a result without a null of the same shape is not a result.**

Layer 0 is the exception in every head-level measure, consistent with it being the only layer whose input does not depend on upstream choices. The rest of this section asks how far that exception goes and what it is made of.

5.7.1 Replication on GPT-2

`gpt2_l18/stc_h20_cat128` and its seed-43 twin differ only in seed: 5×20 heads, $k=256$, `ConcatDictBlock` at $d_{\text{head}}=128$, both at 371,900 steps, scored on the held-out tail. Each measure is run across seeds and, as the positive control, against the seed-42 run’s own checkpoint at step 350,000:

layer	partition NMI, seeds	control	contribution cos, seeds	control
0	0.359	0.976	0.171	0.956
1	0.188	0.766	0.017	0.758
2	0.186	0.602	0.007	0.588
3	0.186	0.538	0.005	0.516
4	0.186	0.510	0.004	0.483
all	0.221	0.679	0.041	0.660
null	0.179	0.179	0.0017	0.0017

Table 10: GPT-2 concat seed pair (`partition.py`, 32,768 rows; `headcontrib.py`, 8,192 rows, null maximum 0.0030).

Only layer 0 reproduces, at under a fifth of the control, and its heads match into the other seed’s layer 0 twenty times of twenty on both measures. Layer 1 keeps its identity — 19/20 and 20/20 of its heads land in the other seed’s layer 1 — while agreeing on almost nothing it separates or writes; layers 2–4 lose even that and scatter across the deep layers. The two measures agree layer for layer, so heads neither carve alike while writing differently nor the reverse. The partition null is high because plug-in NMI between two 256-way partitions of 32,768 rows is bias-dominated, so it cannot resolve the deep layers; the contribution null is tight, and there they sit two to four times above it at about 1% of the control — real, and negligible. The excess over null is larger than on SONAR ($24\times$ against $5\text{--}11\times$), almost all of it in layer 0.

5.7.2 What makes layer 0 reproduce: gain, atoms, and depth

Layer-0 heads are the largest and the most reproducible, so “layer 0 reproduces” might be “large heads reproduce”. `headcontrib.py` measures a head’s size as its centered contribution energy over the whitened target variance, and splits it exactly: a head’s contribution on a row is its layer’s input gain times the decoded atom it selected, so its size is gain^2 times atom spread^2 (the usage-weighted RMS distance of its decoded atoms from their mean); on both stacks the product reproduces the measured size to a log correlation of 1.000.

Layer 0’s larger contributions are all gain. It receives the raw input — GPT-2 activations of norm ~ 122 against residuals of ~ 60 ; unit SONAR embeddings against residuals of 0.36–0.59 — while its atoms are no larger than layer 1’s on either dataset, and on SONAR smaller than those of layers 1–3 and equal to layer 4’s. Gain is one number per layer per row, shared by all of a layer’s heads, so across layers “size” is the layer itself under another name. At equal atom spread, layer 0 still reproduces $14\times$ better on GPT-2 and $23\times$ better on SONAR (log agreement on log spread and a layer-0 indicator, R^2 0.979 and 0.941).

Against the flat arm at matched size. Within a stack, head size and layer do not overlap, so their effects cannot be separated there. The flat arm breaks the confound: all its heads see the raw input, with gain 1 and nothing downstream. On SONAR its own agreement tracks log

layer	GPT-2 concat			SONAR stack		
	agreement	gain	atom norm (10^{-4})	agreement	gain	atom norm
0	0.171	122.2	4.6	0.080	1.000	0.043
1	0.017	67.9	5.3	0.005	0.588	0.054
2	0.007	62.3	4.4	0.004	0.490	0.052
3	0.005	59.2	3.7	0.003	0.417	0.051
4	0.004	57.2	3.2	0.003	0.359	0.043

Table 11: Head agreement (matched centered contribution cosine) and the two factors of head size, medians over each layer’s heads; atoms decoded and whitened. SONAR: `ste_h76_init01` against `ste_h76_i01_s43`, 4,096 rows.

head size at $r = 0.876$ — from 0.008 for its smallest heads (0.057–0.060% of target variance) to 0.150 above 0.071%, and 0.78 and 0.56 for its two largest — so size matters a great deal on its own, and since its gain is constant it is an atom-magnitude curve. But the stack’s heads fall far below that curve at every depth, layer 0 included:

stack layer	size, median	stack agreement	flat heads at that size ($\pm 10\%$)	gap
0	0.31%	0.080	0.67 ($n=2$)	$\sim 8\times$
1	0.16%	0.005	0.12 ($n=1$)	$\sim 23\times$
2	0.11%	0.004	0.19 ($n=3$)	$\sim 49\times$
3	0.073%	0.003	0.059 ($n=115$)	$\sim \mathbf{19\times}$
4	0.039%	0.003	below the flat range	—

The stack costs reproducibility at matched size, at every layer. Layer 0 sees exactly the flat arm’s input and still reproduces several times worse than an equal-size flat head: in the stack, what a layer-0 head should carve depends on what the layers after it do with its error, so the input alone no longer pins it. Deeper layers lose more. This is the reproducibility face of the entanglement result (Section 5.5): the cascade that makes a single-head intervention leak into 59% of the other heads also leaves each head underdetermined by the data. Read against the flat curve, the gain difference between layers 0 and 3 accounts for roughly a third of layer 0’s lead in log terms; only layer 3’s size match is solid — flat heads are almost all small, so layers 0–2 rest on one to three heads each — so the fraction is rough and the direction firm. The matched-size comparison is SONAR-only; GPT-2 has no flat arm.

Within layer 0 the atoms decide. SONAR’s layer-0 gain is identically 1, so its within-layer size variation is pure atom magnitude, and there log agreement tracks atom norm at $r = 0.62$ and spread at 0.64 (cosine against log size, 0.91); GPT-2’s layer 0 shows 0.17 and 0.42. Past layer 1 the atoms predict nothing on either dataset. Spread and norm are one quantity here — their ratio is 0.99–1.08 in every layer, so a head’s usage-weighted mean decoded atom is near zero: the non-negative dictionary’s offset is cancelled by the decoder before it reaches the output.

5.7.3 How entries share a layer

An entry writes only where it wins its head, and then writes its layer’s gain times its decoded atom, so its energy over the data is $\mathbb{E}[\mathbf{1}\{\text{selected}\} \text{gain}^2 \|\text{atom}\|^2]$. Per layer, over 32,768 held-out rows (`entryshare.py`):

run	layer	Gini	top 10% carry	max/median	from atom norm	within heads
GPT-2 concat, $k=256$	0	0.587	42%	38.7	76%	60%
	1	0.155	16%	30.3	44%	100%
	2–4	0.11–0.12	14%	~ 2	$\sim 40\%$	100%
SONAR stack, $k=32$	0	0.326	32%	46.3	92%	83%
	1–3	0.06–0.10	12–14%	1.5–1.9	16–24%	$\sim 100\%$
	4	0.195	18%	3.2	77%	87%
SONAR flat, 1×380	—	0.190	23%	64.1	90%	76%

Table 12: Inequality of entry energy within a layer. “From atom norm” is the covariance share of the variance of log entry energy (the remainder is usage; gain conditional on selection contributes ~ 0). “Within heads” is the share of that variance not explained by which head an entry is in. No entry is dead in any layer.

Layers 1–3 are near-equal; layer 0 is not, and its atoms are why. Deep entries have an effective number of 95–99.5% of the total and the largest is about twice the median; what inequality they have is mostly usage, itself mild (CV 0.14–0.18). Layer 0’s usage is just as even — on SONAR every entry fires 2.1–4.5% of the time against $1/k = 3.1\%$ — but its atom norms vary five to ten times more (CV 0.34–0.52 against 0.02–0.10), mostly inside heads. The flat arm shows the same pattern more mildly, so it belongs to classifying the raw input. SONAR’s last layer is unequal for the other reason: some atoms have shrunk almost to nothing (smallest/median 0.017), the inert final layer of Section 7.3. Whole layers are unequal by gain: layer 0 holds 67% of entry energy on GPT-2 and 49% on SONAR, layer 4 4–5%.

Where the large atoms sit differs completely between the models (`bigatoms.py`):

	GPT-2 concat, 20 heads	SONAR stack, 76 heads
largest atom / head median	3.7 (1.4–5.1)	2.0 (1.5–4.4)
atoms $> 3 \times$ layer median	58, in all 20 heads	11, in 4 heads
atoms $> 5 \times$	1	5, all in head 54
largest head’s energy share	15.5% ($3 \times$ uniform)	9.2% ($7 \times$ uniform)
Spearman(atom norm, usage) in heads	+0.81	+0.27

On GPT-2 every head has a graded tail — its eighth-largest atom is still twice its median — and the large atoms are its frequent outcomes ($1.6 \times$ uniform usage). Agreement is fairly even across heads (0.135–0.235) and only loosely tied to atoms (Spearman 0.49 with energy share, 0.34 with the largest atom). On SONAR the large atoms are a few heads’ property, and fire at the uniform rate: large, not frequent. Head 54 holds 8 of the 11 atoms above $3 \times$ and all 5 above $5 \times$, carries 9% of the layer, and reproduces at **0.844** against 0.249 for the next head; across heads, agreement tracks the largest atom (Spearman 0.62) and how peaked the head’s profile is (0.62), not its typical atom (-0.14). That resolves the within-layer-0 atom effect into a few heads with outsized atoms — but not the layer-0 lift: without head 54 the layer-0 mean falls only from 0.080 to 0.070, still twenty times the deep layers. Head 54 is also the index of the topic/genre head of the shipped-init `ste_h76` (Section 5.8), which shares its seed; the same head may be recurring, unverified.

The large atoms are distinct directions: different heads’ largest atoms are near orthogonal (median $|\cos|$ 0.022, below random atom pairs). On SONAR they lean into the data’s top ten

principal directions (9% of their energy against 3% for a typical atom), consistent with cross-seed agreement being the data’s dominant directions; on GPT-2 they lean the other way (4% against 13%). One seed pair per dataset throughout.

5.8 Does a labeled variable land in one head?

Held-out language identity over 86 languages (dense ridge ceiling 0.721, chance 0.012):

arm	shape	best-head NMI	less null	probe $m=1$	$m=32$
<code>ste_h76</code>	5×76	0.269	0.261	0.165	0.378
+ collinearity setpoint	5×76	0.287	0.279	0.139	0.318
– mean-entropy bonus	5×76	0.085	0.078	0.038	0.322
+ head independence	5×76	0.199	0.189	0.097	0.383
flat	1×380	0.277	0.276	0.102	0.257
+ collinearity setpoint	1×380	0.247	0.246	0.080	0.271
softmax reference	5×32	0.055	0.043	0.036	0.275

Table 13: `headlang.py`, shipped initialization. The attainable NMI for a 5-bit head against the 6.43-bit label is 0.875.

No head is a language variable in any arm: the best reaches 0.28 of an attainable 0.875, and no head-health lever changes that. A $k=32$ head cannot hold 86 languages at all ($\log_2 32 = 5 < 6.43$ bits), so the clean test needs $k \geq 128$; the gap is wide enough that the verdict stands. **The hard code is consistently the more concept-aligned**, six times the softmax reference on null-corrected best-head NMI. The stack beats the flat arm on probe accuracy at every head budget (0.378 vs. 0.257 at $m=32$). (`summary.json`’s `info_share_best` favours the flat arm, but it divides by summed plug-in MI whose per-head bias differs eightfold between the shapes; do not read it across architectures.)

Script, tags, conjunctions. Script is not the variable either (best-head NMI 0.191 of a 0.671 ceiling; the same head, L0 h50, is best for both). A single tag is a strong but permissive detector: the best cell per language reaches recall 0.74–0.98 at FPR 1.6–2.2%, but no language reaches precision 0.8. Conjunctions of top cells from distinct heads do not fix it: F1 peaks at $m=2$, 7% above a single tag, and the best precision is 0.598 at 4.8% recall. Recall combines nearly independently (0.234 against 0.262 predicted) but false positives do not (precision 0.43 at $m=2$ against 0.83 if independent): many heads send the same confusable rows to the same wrong language — Finnish with Estonian, Japanese with Korean. The code encodes language *neighbourhood* rather than identity.

Why, from the embedding geometry. Language explains 6.51% of embedding variance and script 2.26%, and corrected for group count (`headeta.py`) language is a *stronger* partition than 90% of the heads ($51\times$ its random baseline, against $5\times$ for the median head and $235\times$ for the best). So neither salience nor capacity is the obstacle. Languages are weakly separated regions, not directions: within-language mean cosine is 0.3596 against 0.3160 between, and the shared mean direction accounts for 0.3178 of that, leaving a +0.044 excess, 0.30 sd of the pair distribution. A partition code cuts along local residual variance, and at 0.30 sd those cuts pass through the clusters rather than around them. Layer-4 heads are near-random partitions ($2\times$ baseline), which is Section 7.3’s inert final layer seen from a different direction.

What the strongest head holds. L0 h54, the $235\times$ head, is not a language head (NMI 0.149; 16.1 distinct languages among an entry’s top-20 rows). Decoded as deviations from the head mean, its entries read as topic and genre — crime news, geopolitics and parliament, encyclopedic place description, manufacturing, marketing, legislative text, entertainment. Contrastive auto-interp (`cacts`, null 0.273) agrees and adds to it: genre/topic descriptions (16 entries) score F1 0.523, multilingual or code-switched passages (8) 0.476, text artifacts such as duplication and mojibake (3) 0.395, script or region (5) 0.394. All four beat the null. Topic dominating sentence-embedding variance explains both the head’s top η^2 and its low language NMI. Per Section 5.7, this describes one head of one seed — though index 54 is also the fixed-init stack’s one strongly reproducible head (Section 5.7.3).

6 Hard codes on GPT-2 small: entries in activation space

A second line of straight-through runs (`experiments/gpt2`, from the `headline` branch) takes the hard code to GPT-2 small and adds the measure an embedding substrate lacks: the fraction of GPT-2’s loss recovered when the reconstruction is spliced back into the model, $(CE_{\text{ablate}} - CE_{\text{recon}})/(CE_{\text{ablate}} - CE_{\text{clean}})$ with the residual mean-ablated as the floor.

Setup, and what is not comparable. Activations are the residual stream *entering* block 8 (Section 5.7’s GPT-2 runs use the stream leaving it), mean-centred and scaled to unit mean square, on FineWeb; FVU_w here is unwhitened. These numbers therefore do not compare with the other GPT-2 results in this document, only with each other and with the SAE baselines trained on the same cache. Held-out evaluation uses documents disjoint from training: 520k tokens for the 40M-token runs, 65k for the one-epoch ones. The runs use three options this branch does not implement — `direct`, `resid_first` and `gain_clip`, below — so they reproduce only on `headline`; its private heads are this branch’s `ConcatDictBlock`, and its fibers and head dropout have been ported.

The design. Relative to Section 5, three changes. *Direct entries*: dictionary entries are signed vectors in activation space itself, with no latent dictionary space and no decoder, so every class is literally a steering direction. *Gain-shape on every layer*: layer 0 too classifies the unit direction of what it corrects (`resid_first`), and each layer’s contribution is scaled by the observed residual norm, clipped at $\|X\|$, which makes the model positively homogeneous, $f(cX) = cf(X)$. *Annealed straight-through selection*: argmax forward, softmax backward, the temperature annealed $1 \rightarrow 0.03$ over the first 60k of 156k steps. With 5×32 heads of 32 labels the code is 800 bits per token with no continuous coefficients.

A discrete 800-bit code recovers more of GPT-2’s loss than a $k=32$ SAE at nearly the same FVU_w : 97.6% against 96.2%, at 0.195 against 0.189. Across architectures, then, FVU_w does not order downstream loss — the SAE’s slightly lower error costs the model more — where within the Ontologizer family on SONAR text fidelity was a monotone function of FVU_w (Section 4.9). Raising the labels per head to 64 (960 bits) passes the SAE on both measures. Every layer contributes (prefix FVU_w $0.472 \rightarrow 0.347 \rightarrow 0.278 \rightarrow 0.231 \rightarrow 0.195$ for the direct code, $0.451 \rightarrow 0.186$ for private heads), label usage stays near uniform in every layer (imbalance 0.01–0.16 bits of 5; 0.30 at the private code’s last layer), and the observed gain is close to the best per-sample scale for each layer’s contribution (optimal rescale 0.89–1.09, coefficient of variation 0.05–0.08).

model	entries	code per token	FVU _w	loss recovered
direct + gain + STE	signed, in $\mathbb{R}^{768}$	800 bits	0.195	97.6%
private heads + gain + STE	48-dim blocks, shared decoder	800 bits	0.186	97.5%
private heads, $k=64$	48-dim blocks, shared decoder	960 bits	0.164	97.9%
top- k SAE, $k=32$, $m=5120$	—	32 coeffs + ~ 394 bits	0.189	96.2%
top- k SAE, $k=128$, $m=5120$	—	128 coeffs + ~ 1580 bits	0.116	98.6%
soft Ontologizer (1 epoch)	—	4960 mixture weights	0.016	99.85%

Table 14: GPT-2 small, residual entering block 8. Ontologizers: one pass over 40M tokens, one seed. SAEs: 10 epochs over 10M tokens with `sae.py`’s unwhitened objective. Clean CE 3.806, mean-ablated 8.283; the direct code reaches 3.914.

What each change buys. One-epoch runs (10M tokens) isolate them:

variant (800 bits unless noted)	FVU _w	loss recovered
STE alone (latent dictionary + decoder)	0.327	93.3%
+ gain	0.290	94.3%
+ gain, private heads	0.288	94.2%
+ gain, direct entries	0.217	96.95%
seed 43	0.219	96.85%
$k=64$ (960 bits)	0.186	97.6%
$k=128$ (1120 bits)	0.165	97.9%

Gain-shape and direct entries each help, and direct entries most. Their advantage is partly speed: over 40M tokens private heads close the gap and finish slightly ahead (0.186 against 0.195). Without direct entries and gain, more labels per head collapse rather than help (FVU_w 0.75 and 0.80 at $k=64$ and 128), and the other straight-through configurations froze their last layer at 4.8–5.0 bits, every head constant; the direct code’s deepest layer never froze. The base result reproduces across seeds to 0.002 FVU_w. Annealing matters: straight-through at a constant $T=1$ reaches 0.50 and a scheduled soft-to-hard mix 0.39. Sparse encoders in front of the classifier never beat the dense one: at one epoch, 32- and 640-wide front-ends reach 0.25–0.29 against the dense classifier’s 0.217, and a 4096-wide, 64-active encoder per head with dead-latent resampling reaches 0.225 against 0.195 for the dense front-end on the same 40M tokens, with 88% of its layer-1 latents dead at no cost to that layer.

Fibers. A rank- r local basis per entry, with coordinates read linearly from the unit residual (`fiber_rank`; ported to this branch), turns the code into a discrete base point plus r numbers per head. “Base only” is the same model with the fibers removed, and measures whether the discrete code still means what it did:

variant (1 epoch, direct + gain + STE)	extra numbers	FVU _w (base only)	loss recovered
base	0	0.217	96.95%
rank-4 fiber, fiber dropout 0.5	640	0.059 (0.274)	99.4%
rank-8 fiber, fiber dropout 0.5	1280	0.014 (0.304)	99.9%
rank-4 fiber, joint norm cap 0.2	640	0.180 (0.227)	97.6%
rank-1 radial fiber (gain per entry)	160	0.148 (0.669)	98.5%
frozen base, rank-4 fibers trained after	640	0.016 (0.215)	99.9%
unconstrained fibers	—	— (~ 10)	—

Fibers recover far more than the SAE’s FVU_w advantage, but they carry hundreds of continuous numbers per token — more than the $k=32$ SAE’s 32 coefficients — so this is not a matched-capacity comparison. The question they raise is what keeps the discrete base honest. Unconstrained, the fibers take over (base-only $\text{FVU}_w \sim 10$); fiber dropout holds the base at 0.23–0.30 whatever the rank; a joint norm cap guarantees it at the cost of most of the fibers’ value. **Training the fibers after freezing the base is best on both counts:** the base is untouched by construction (0.215) and the fibers reach the reconstruction of unconstrained joint fibers. The mechanisms used in joint training were suppressing the fibers, not protecting the base.

Seed stability, replicated. Matching heads across two seeds by partition NMI (`seedstab.py`), the partitions agree only in layer 0 (NMI 0.09 against 0.05 for unrelated heads within one run) and not at all in layers 1–4. With direct signed entries, gain-shape from layer 0, a different hook and a different objective, this is the pattern of Section 5.7: which labels share a head is seed-dependent past the first layer.

7 Dictionary geometry and the non-negativity constraint

7.1 Row collinearity is mostly gauge

Within-head row cosine (`cossim_k, rowcos`) is computed on `dicts()` in the e -dimensional dictionary space. The decoder is $d \times e$ at full rank, so $e - d$ of those dimensions are its null space, and on the SONAR straight-through arm atoms put 74–78% of their energy there, where nothing they do reaches the output and no gradient reaches them. Decoded into output space the same statistic reads 0.023 at layer 0 and -0.015 to -0.020 below it, against 0.12–0.29 in e .

Dictionary statistics should be computed after decoding.

On GPT-2 layer 8 ($e=1536$, $d=768$) four tempting readings of the null-space mass each failed a check: head null components add coherently ($3.3\text{--}11\times$ over the cancelling expectation), so they are not interference debris; the fraction does not track feature noise; the per-atom visible fraction is flat at 22% across training and the decoder’s row space rotates only $2.6\text{--}3.5^\circ$, so it is not a swap space; and a mixed-sign visible component needs no non-negative “lift”. Two baselines also had to be corrected: for $|\mathcal{N}(0,1)|$ atoms the chance visible fraction is 18.26% (not 50%) and the chance alignment with **1** is 63.7% (not ~ 0), so the trained dictionary is *less* null-heavy than chance; and a reported 49% of reconstruction gradient in $\ker(D)$ was a coordinate artifact, $3e-8$ with respect to $|W|$. How large “the null part” is depends entirely on the norm asked for: 2.43% of squared L2, 23.20% of L2, 56.55% of L1.

7.2 The orthant forces a collinear dictionary

Non-negativity binds through the Gram, not the null space: two non-negative vectors have $\langle a, a' \rangle \geq 0$ exactly, so for independent $|\mathcal{N}(0,1)|$ entries the chance pairwise cosine is $2/\pi = 0.637$, with minimum $+0.58$ over 4000 draws; signed draws give 0.000. **--signed** drops the `abs()` in `DictBlock.dicts`. On GPT-2 layer 8 with the straight-through arm’s shape, crossed with dictionary width, at convergence (371,900 steps; `dictgeom.py`):

The forced collinearity is nearly free in reconstruction. At $e=1536$, `abs` runs at a third of the signed effective rank for 2.4% of FVU_w . The reconstruction cost is an *interaction*: narrowing e costs $+0.1\%$ signed and $+14.8\%$ under `abs`. So the wide dictionary was buying

cell	c (row cosine)	eff. rank	FVU _w	at step 93k
signed, $e=1536$	+0.0009	27.37	0.3363	0.5362
signed, $e=768$	+0.0019	26.95	0.3367	0.5410
abs, $e=1536$	+0.3038	9.15	0.3445	0.5964
abs, $e=768$	+0.4543	5.41	0.3954	0.6532
concat, $d_{\text{head}}=32$	+0.5353	3.30	0.5140	0.8545

Table 15: Non-negativity $\times$ width on GPT-2 layer-8 residuals. No collinearity penalty is active in any cell.

room for the constraint, not representational capacity — which tempers Section 3.1.4’s claim that head-disjointness is what $e > d$ buys. The prediction made in advance — three cells alike and **abs**/768 worse — is what convergence shows, but at a quarter of the run **signed** led by 10.1% at $e=1536$ and the prediction looked falsified; **abs** at the wide width is merely slower. A geometric reading via Wendel’s theorem [27]: m random columns lie in a common halfspace with probability $2^{1-m} \sum_{i < d} \binom{m-1}{i}$, exactly $1/2$ at $m=2d$, so $e=1536$ sits on the threshold where a random decoder’s cone $\{Da : a \geq 0\}$ covers the output space at all.

What removing **abs** buys is therefore dictionary geometry rather than error: a decorrelated, near-full-effective-rank dictionary at matched reconstruction — roughly what the collinearity setpoint exists to buy. Signed atoms give up two things: subtractive features are harder to read as “this concept is present”, and the $|F|_1 \geq h$ floor under **norm.rows**, which rests on non-cancellation, disappears.

Concatenated heads. `ConcatDictBlock` at $d_{\text{head}}=32$ gives each head a (768, 32) decoder block: 32 non-negative generators in a 32-dimensional subspace, a simplicial cone, and the per-head null space is zero. The floor on a head’s mean row cosine is $(k/n - 1)/(k - 1)$ with $n = d_{\text{head}}$, which is 0 at $n=k=32$, so concat faces no raised floor; it faces a collapsed tradeoff, since $c=0$ at $n=k$ needs one coordinate per atom. It took the correlation, the lowest effective *and* numerical rank (rank99 17.1 of 32, against 30.2 summing), and the worst error. Its extra cost is reachability, the one place the cone frame binds; it is not explained by c alone (at step 93k it had lower c than **abs**/768 and much worse error).

7.3 Selection rule, training duration, and the last layer

Softer codes do not raise the dictionary’s rank. On the SONAR selection sweep, numerical rank is full (rank99 30.7–31.8 of 32) in all ten arms, and effective rank varies 7-fold but is not monotone in softness, with the two families ($\pm$ mean-entropy bonus) ordering oppositely at convergence. Tracing each arm over training resolves it: **training duration dominates the selection rule by $3.8\times$** (mean c falls $0.750 \rightarrow 0.217$ from step 10k to 369.5k, while the spread across rules at a fixed step averages 0.140), and the orderings cross near step 100k:

step	top2	top4	top8	top16	softmax	spread
10,000	0.795	0.788	0.774	0.743	0.648	0.147
50,000	0.658	0.639	0.612	0.588	0.519	0.139
100,000	0.459	0.418	0.406	0.426	0.435	0.053
150,000	0.329	0.275	0.256	0.303	0.370	0.114
369,500	0.218	0.151	0.141	0.215	0.359	0.218

(`_shm` family.) The soft arm’s c plateaus once reconstruction is good, while hard arms keep paying collinearity down long after their own error flattens (**top2**: FVU_w -7% , c -34% from 150k to 369.5k). Under a hard code a head’s output *is* one of its rows, so collinear rows make the output independent of which entry wins; collinearity costs a hard code information directly, and a soft code, with continuous coefficients, much less. At convergence the softer code has the *more* collinear dictionary. Matching arms on FVU_w instead of step is the mirror confound (at FVU_w 0.40 **top2** is at step ~ 234 k, the soft arms at 23–43k). One seed per cell.

The last layer is inert once the error is small. Collinearity falls monotonically with depth in every hard arm, but soft arms spike at the last layer (**softmax_shm** layer 4: $c=0.799$, effective rank 1.54). `lastlayer.py` ablates the final layer’s heads two ways: uniform (emit the mean atom) and zero (emit nothing):

arm	select	L4 c	L4 eff	prefix FVU_w before L4	L4 adds	zero-ablate
top1	top1	+0.246	10.60	0.7511	+4.3%	+4.5% worse
top2_shm	top2	+0.188	14.89	0.2378	+19.7%	+24.5% worse
top8_shm	top8	—	—	0.0304	+5.5%	+5.8% worse
top16_shm	top16	+0.433	3.46	0.0043	+0.0%	+0.0%
softmax	softmax	+0.263	6.00	0.0023	-1.2%	-1.1% better
softmax_shm	softmax	+0.799	1.54	0.0007	-1.9%	-1.8% better

The last layer’s contribution tracks the error left before it, not its collinearity: **softmax**’s layer 4 is not collapsed ($c=0.263$) and is just as useless. Below a prefix FVU_w of about 0.005 the final layer is inert or mildly harmful — which accounts for **resid_nc**’s “mildly harmful” layer 4 (prefix 0.0051) without appeal to its frozen heads. The two collapses end differently: **top16_shm**’s layer 4 shrank its norms to 4% of the other layers’ and emits nothing; **softmax_shm**’s is collinear at near-equal magnitude, emits a near-constant vector, and removing it helps — a small overfit, not a useful bias.

8 Formal reading: heads as causal variables

The steering frame has a natural formalization in the causal-abstraction vocabulary, and stating it makes several instruments in the battery estimators of named quantities rather than ad hoc scores.

Definitions. Read the Ontologizer as a structural causal model: each head (l, h) is a candidate high-level *variable* with value space $\{1, \dots, k\}$ (soft state: its simplex), the residual stack is the causal graph (with residual forwarding, layer $i+1$ ’s classifier reads the residual written by layers $\leq i$, so every earlier head is a parent of every later one), and **withArgs** set-interventions are the do-operator: $\text{do}[(l, h)=k']$ forces the head’s assignment while leaving all non-descendants untouched by construction. The abstraction map τ sends an embedding x to its per-head assignment profile $P(x)$. Under this reading: **steerembed.py**’s realization rate is an estimator of whether an *input-space* intervention implements $\text{do}[(l, h)=k']$, and its collateral of how far it also intervenes on other variables; **steerfid.py**’s cycle-consistency gain estimates *interchange-intervention accuracy* in the closed loop $\tau \circ \text{encode} \circ \text{decode}$, bounded above by the $\sim 12\%$ round-trip survival of Section 4.9; and seed-to-seed matching is *empirical identifiability*. What

the softmax leaves unidentified in principle is the per-head entry order (permutation) and any logit gauge the temperature absorbs; what must therefore be reproducible for the variable reading to hold is the partition itself and the decode directions up to relabeling. Section 5.7 measured exactly that on the hard code, on SONAR and GPT-2. Past layer 0 the variables are identified at the level of the layer, not the head — and on GPT-2 not even that beyond layer 1. Layer 0 is partially identified, with SONAR’s identification concentrated in a single large-atom head.

Why the cascade costs identifiability. The stack’s reproducibility deficit at matched head size (Section 5.7.2) and its intervention collateral (Section 5.5) are one fact under this reading. A head’s value is a function of its own input, and every head past layer 0 takes as input the residual that upstream heads leave; a different upstream solution changes what any downstream variable is *about*. Even layer 0, whose input is the data, is pinned only partly: its heads are trained jointly with everything downstream of them, so which partition of the raw input is best depends on what the later layers will do with the error. A flat layer has neither dependency, and its heads are both more reproducible at equal size and independently addressable.

Metric hygiene. Claims about decode-direction geometry should not depend on an arbitrary choice of inner product. The contrast-set result does not: under the training objective’s inverse-variance metric the Ontologizer’s within-head repulsion strengthens (-24.4 vs. -17.6 raw) and both rivals’ signatures are unchanged. Two further rules emerged. Measure in output space, not in the dictionary space, whose null-space component is unconstrained (Section 7.1). And separate a layer’s gain from a head’s atoms before attributing anything to head size: across layers, size is the gain, and the gain is the layer (Section 5.7.2).

Composition. A variable reading earns little if interventions on two heads interact uncontrollably. Measured additivity of decode deltas on `resid_nc` (`compose.py`, whitened metric, medians over pairs):

stratum	n	cos	rel. err	note
same-layer L0	28	0.959	0.350	4 downstream layers
same-layer L1	28	0.947	0.354	3
same-layer L2	28	0.967	0.259	2
same-layer L3	28	0.996	0.092	1
same-layer L4	28	1.000	0.019	0 (= numerical floor)
cross-layer (earlier = L0/L1)	42	0.97–0.98	0.20–0.23	
cross-layer (earlier = L2/L3)	18	1.00	0.02–0.04	

Three observations. (i) Joint interventions point where the summed singles point everywhere (median cosine ≥ 0.95 in every stratum): **withArgs** interventions compose approximately additively. (ii) The non-additive component is entirely attributable to downstream reclassification: it decays monotonically with the number of layers below the intervention and hits the numerical floor when there are none (the measured 0.019 is float32 rounding amplified by the decoder cancellation, $\sim 3 \times 10^{-6}$ in float64). (iii) Cross-layer pairs compose *better* than same-layer pairs at matched depth: a set-intervention is immune to upstream perturbation, so cross-layer interaction is one-way. Note the contrast with Section 5.5: **withArgs** sets a head directly and composes well, while an *input-space* push reaches every head and disturbs most of them. The

downstream reclassification that is a 5–35% correction to a set-intervention is the dominant effect of an input-space one.

9 Preliminary conclusions

1. **Contrast-set structure is real, and it is what the architecture buys.** Trained softmax heads’ decode directions repel (96% of heads below null), and hard-code heads’ decoded entries are orthogonal to slightly repelling: each head holds well-separated alternatives. This is invisible to reconstruction metrics and to per-latent probes.
2. **A discrete ontology is possible, but not at the softmax architecture’s size or with its readout.** The softmax model’s argmax is not its code ($77\times$ above the 800-bit floor for `resid_nc`, nine orders of magnitude on `softmax_shm`). A straight-through arm with $2.4\times$ the heads makes the discrete code the model, and at 1900 index bits and no coefficients beats converged SAEs at matched index bits that also send 160 coefficients (0.154 vs. 0.258–0.274), and an L1 SAE spending 519 coefficients and $3.4\times$ the bits (0.220), within $3.1\times$ of its own rate floor. It needed a temperature matched to the logit scale, relative logit noise, a rescaled entropy bonus, and an initialization fix worth a third of its error.
3. **The discrete code holds up downstream.** On GPT-2 small, an 800-bit hard code with signed entries in activation space and gain-shape on every layer recovers 97.6% of GPT-2’s loss, more than a $k=32$ SAE at nearly the same FVU_w (96.2%), and 64 labels per head pass it on both measures. Across architectures FVU_w does not order downstream loss. Fibers on top of a frozen discrete base recover almost all the rest (99.9%) while leaving the base exactly as it was, so a discrete ontology and a near-lossless reconstruction need not trade against each other — at the price of hundreds of continuous numbers per token.
4. **Reconstruction is the price sheet, not the verdict, for soft codes.** Unstructured codes reconstruct $\sim 50\times$ better at matched coefficient budgets, and the grouped-simplex constraint alone costs $\sim 30\times$; text fidelity is a monotone function of FVU_w and adds no independent axis.
5. **The residual cascade is measured on both sides of the ledger.** For: $1.79\times$ lower error than a flat layer at equal bits, parameters and 99.9% utilization, sourced in the pairing of each stage’s input with the sample’s own error (not utilization, conditioning, or the gain side-channel). Against: any input perturbation, steering included, reclassifies $\sim 60\%$ of downstream heads, where a flat layer’s heads are independently addressable (97% realization at 9% collateral); and at equal head size the stack’s heads reproduce across seeds several times worse than flat heads at every layer, layer 0 included. The single-layer rival `g160softmax` matches `resid_nc` on reconstruction, but a config-matched softmax Ontologizer reaches $6\times$ lower error (in-sample), so the reconstruction verdict against the stack was checkpoint-specific.
6. **Steering works in embedding space and the text cycle hides it.** The decode direction realizes a target entry on 82% of rows for the softmax model at strength 0.25, against a 3% chance rate, while the text round trip can observe at most $\sim 12\%$ of perfect interventions. The soft code is the more steerable at low collateral; steerability does not

track reconstruction, and the classifier’s own gradient is the right direction only where the input reaches the classifier directly.

7. **Individual heads past layer 0 are not reproducible, and layer 0 only partly.** Seeds agree on the error to four digits. Past layer 0 they agree on nothing that identifies a head — partitions, contributions, subspaces beyond the data’s, or Grams — each tested against a same-shape null, on SONAR and on GPT-2, and again under the direct-entry design of Section 6. Layer 0 reproduces at under a fifth of a same-run control. Its lead over the deeper layers is not atom magnitude: its atoms are no larger, and its larger contributions are its gain, which is a property of the layer. Within layer 0, heads with outsized atoms are the reproducible ones — on SONAR almost entirely one head. This remains the most consequential result for the steering frame: a head that is not the same head on the next run cannot be named, and every per-head claim here inherits that.
8. **Entries contribute near-equally except where the input is raw.** In layers 1–3 every entry carries close to its share (Gini 0.06–0.15, no dead entries); layer 0’s entries are unequal (Gini 0.33–0.59) because its atom norms vary widely inside heads, while usage stays near-uniform. The flat arm shows the same pattern, so it belongs to classifying the raw input.
9. **No head is a labeled variable.** The hard code is six times more language-aligned than the softmax reference, yet its best head reaches 0.28 of an attainable 0.875 NMI, with false positives shared across heads: the code encodes language neighbourhood, because languages are weakly separated regions of SONAR space rather than directions. The strongest head encodes topic and genre.
10. **Per-latent lenses mis-measure classifier codes.** k -sparse probing ranks the Ontologizer worst while partition-level analysis finds verified causal language heads; detection F1 is $\sim 80\%$ explained by code density, and the Ontologizer sits on the SAE density trend. Parameter-space decodes are instances, not descriptions, and must be taken as deviations from the head mean to be legible at all.
11. **Non-negativity is cheap in error and expensive in geometry.** It forces a collinear, low-effective-rank dictionary (a third of the signed rank) for 2.4% of FVU_w at sufficient width; at a square decoder the cost becomes 14.8%. Dictionary statistics computed in the dictionary space rather than after decoding are mostly gauge.
12. **Every measure in this architecture needs a null of the same shape.** Non-negativity and the layer-dependent gain put structure into every floor. Three of four seed-reproducibility measures, the HSIC head dependence, the null-space readings, and the apparent size effect on reproducibility each reversed or dissolved under the right control.

10 Follow-up investigations

Ordered by relevance to the steering goal.

- **Head identifiability first.** Before naming heads, make them reproducible or establish which are. Candidates: partition-level seed stability on the softmax Ontologizer and

`g160softmax`; whether large-atom heads’ reproducibility can be extended by training pressure; whether the $k=2$ arm’s one-bit heads reproduce better than 32-way ones; whether SONAR’s head 54 at the fixed initialization is the shipped-init run’s topic head (same seed, same index); and a GPT-2 flat arm, so the matched-size comparison of Section 5.7.2 has a second dataset.

- **Bring the GPT-2 hard code onto this branch, then to SONAR.** Section 6’s runs need `direct`, `resid_first` and `gain_clip`, which only the `headline` branch has; porting them makes those results reproducible here and lets the same design run on SONAR, where its 800-bit code can be compared with the straight-through arm’s 1900 bits. Direct entries also make steering literal — a set-intervention adds $\|n\|(g_{\text{new}} - g_{\text{old}})$ at one layer — and that has not been measured.
- **Fibers on a frozen base.** The two-stage recipe kept the base exact and matched unconstrained fibers’ reconstruction; it should be the default, scored at matched continuous capacity against SAEs (rank-1 radial fibers already sit at 160 numbers per token).
- **Remeasure on the fixed initialization.** Steering, language, gain and blend ablations all ran on the shipped initialization, which was 1.84×10^8 off at step 0 and left 10% of the code unused.
- **The steering overlay at scale.** `steerembed.py` with SAE decoder rows, bilinear eigenfeatures and supervised diff-of-means vectors alongside Ontologizer directions; per-entry effect consistency across contexts; and the flat arm’s near-perfect addressability and better reproducibility weighed against its $1.79\times$ reconstruction cost.
- **Task reconstruction.** Recover what each head classifies: partition-vs-label-library MI (language at $k \geq 128$, script, length, formatting/noise, unsupervised topics); *sibling-contrastive* descriptions (positives where entry k wins within head h , background where its siblings win), validated by blinded assignment accuracy vs. $1/k$; and FLORES parallel-text invariance to split content heads from form heads. Run identically on Ontologizer heads and `g160softmax` groups, and only on heads that pass the identifiability bar.
- **Hierarchy as MoE routing.** Conditional MI between successive layers’ partitions tests whether depth builds compositional tasks; `forward="labels"` at the fixed initialization tests the pairing account from the other side.
- **The collinearity setpoint’s mechanism.** The one lever that buys steerability acts on a statistic that is mostly gauge; repeat it on the decoded statistic, and on signed dictionaries.
- **Logit-scale control for straight-through training.** Normalize logits per head, or control T against a spread/ T setpoint, in place of the hand-matched temperature.
- **Fresh-encoded eval set.** New runs hold the tail out; the softmax Ontologizers do not, and every comparison against them inherits that asymmetry.
- **A top- k sweep at matched bits.** The converged frontier has top- k points only at $k=32$ and 160; a k sweep through the 1,900-bit region would pin the discrete-vs-linear comparison at exactly matched rate.

- **Winner dropout for grouped rungs.** `g160top1`'s 28% dead latents are within-group winner-take-all starvation.
- **Trained truncatability.** A deviation-sparsity penalty at training time would show whether the soft Pareto curve can shift left.
- **Rung composition and frontier fill.** `groups+prefixes` (2b+3), grouped variants at $m=11264$, the mid-range top- k sweep ($k \in \{512, 2048\}$); random-support and nonnegative-LS refit controls.
- **Closed.** HSIC head independence (heads are already independent), decoder sparsification and bias (no structure; $< 1\%$), code-forwarding alongside the residual, input whitening (subsumed by initialization), and entropy-pressure rescaling across shapes (likewise). The `resid_nc` layer-4 freeze is explained by saturation; the two anomalously coherent heads (L2h25, L3h28) remain to inspect.

Reproducibility

The evaluation protocols are root-level scripts in the `ontologize` repository (`sae.py`, `pareto.py`, `headstruct.py`, `headcoh.py`, `splitting.py`, `textfid.py`, `langprobe.py`, `refit.py`, `steerfid.py`, `compose.py`, `autointerp.py`), driven by the `sae_ladder.sh`, `sae_evals.sh`, and `autointerp_run.sh` runners (idempotent, crash-resumable systemd user units; judge calls rate-metered). The straight-through arm and its diagnostics live in `experiments/ste-arm/` (`train_ste.py`, `steerembed.py`, `blendablate.py`, `gainablate.py`, `inputgeom.py`, `dictgeom.py`, `lastlayer.py`, `partition.py`, `headcontrib.py`, `entryshare.py`, `bigatoms.py`, `headlang.py`, `headscript.py`, `headeta.py`, `embedgeom.py`, `decodehead.py`, `make_zca.py`); its notebook `writeup/ste-arm.md` records every run's command and the superseded readings. The GPT-2 hard-code runs of Section 6 live in `experiments/gpt2/` (`train_onto.py`, `diagnose.py`, `seedstab.py`; notes in `DIRECT.GAIN.STE.md` and `results/README.md`, with the two headline runs' configs, diagnostics and loss curves) and need the `headline` branch's code. The curated test suite pins each protocol's statistics against planted-structure synthetics. Artifacts live under `data/out/sonar/` and `data/out/gpt2.18/`. At most three straight-through runs fit concurrently on one 24 GB device.

References

- [1] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. "Product Quantization for Nearest Neighbor Search". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.
- [2] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: <https://arxiv.org/abs/2107.03312>.
- [3] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. 2013. arXiv: 1308.3432 [cs.LG]. URL: <https://arxiv.org/abs/1308.3432>.
- [4] Alex Tamkin, Mohammad Taufeque, and Noah D. Goodman. *Codebook Features: Sparse and Discrete Interpretability for Neural Networks*. 2023. arXiv: 2310.17230 [cs.LG]. URL: <https://arxiv.org/abs/2310.17230>.

- [5] Anish Mudide et al. *Efficient Dictionary Learning with Switch Sparse Autoencoders*. 2025. arXiv: 2410.08201 [cs.LG]. URL: <https://arxiv.org/abs/2410.08201>.
- [6] Zhengyan Zhang et al. *MoEfication: Transformer Feed-forward Layers are Mixtures of Experts*. 2022. arXiv: 2110.01786 [cs.CL]. URL: <https://arxiv.org/abs/2110.01786>.
- [7] Bart Bussmann et al. *Learning Multi-Level Features with Matryoshka Sparse Autoencoders*. 2025. arXiv: 2503.17547 [cs.LG]. URL: <https://arxiv.org/abs/2503.17547>.
- [8] David Chanin et al. *A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders*. 2025. arXiv: 2409.14507 [cs.CL]. URL: <https://arxiv.org/abs/2409.14507>.
- [9] Tue M. Cao et al. *Tree SAE: Learning Hierarchical Feature Structures in Sparse Autoencoders*. 2026. arXiv: 2605.07922 [cs.LG]. URL: <https://arxiv.org/abs/2605.07922>.
- [10] Seongwan Park, Taeklim Kim, and Youngjoong Ko. *Decoding Dense Embeddings: Sparse Autoencoders for Interpreting and Discretizing Dense Retrieval*. 2025. arXiv: 2506.00041 [cs.IR]. URL: <https://arxiv.org/abs/2506.00041>.
- [11] Nick Jiang et al. *Interpretable Embeddings with Sparse Autoencoders: A Data Analysis Toolkit*. 2026. arXiv: 2512.10092 [cs.AI]. URL: <https://arxiv.org/abs/2512.10092>.
- [12] Wonseok Shin and Songkuk Kim. *Aligning Sentence Embeddings to Human Concepts via Sparse Autoencoders*. 2026. arXiv: 2607.00023 [cs.IR]. URL: <https://arxiv.org/abs/2607.00023>.
- [13] LCM team et al. *Large Concept Models: Language Modeling in a Sentence Representation Space*. 2024. arXiv: 2412.08821 [cs.CL]. URL: <https://arxiv.org/abs/2412.08821>.
- [14] John X. Morris et al. *Text Embeddings Reveal (Almost) As Much As Text*. 2023. arXiv: 2310.06816 [cs.CL]. URL: <https://arxiv.org/abs/2310.06816>.
- [15] Adam Karvonen et al. *SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability*. 2025. arXiv: 2503.09532 [cs.LG]. URL: <https://arxiv.org/abs/2503.09532>.
- [16] Gonalo Paulo et al. *Automatically Interpreting Millions of Features in Large Language Models*. 2025. arXiv: 2410.13928 [cs.LG]. URL: <https://arxiv.org/abs/2410.13928>.
- [17] Zhengxuan Wu et al. *AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders*. 2025. arXiv: 2501.17148 [cs.CL]. URL: <https://arxiv.org/abs/2501.17148>.
- [18] Xiangchen Song et al. *Position: Mechanistic Interpretability Should Prioritize Feature Consistency in SAEs*. 2025. arXiv: 2505.20254 [cs.LG]. URL: <https://arxiv.org/abs/2505.20254>.
- [19] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: <https://arxiv.org/abs/2308.11466>.
- [20] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.
- [21] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: <https://arxiv.org/abs/2406.04093>.

- [22] Michael T. Pearce et al. *Bilinear MLPs enable weight-based mechanistic interpretability*. 2025. arXiv: 2410.08417 [cs.LG]. URL: <https://arxiv.org/abs/2410.08417>.
- [23] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley, 2006.
- [24] Maja Popović. “chrF: character n-gram F-score for automatic MT evaluation”. In: *Proceedings of the Tenth Workshop on Statistical Machine Translation*. Ed. by Ondřej Bojar et al. Lisbon, Portugal: Association for Computational Linguistics, Sept. 2015, pp. 392–395. DOI: 10.18653/v1/W15-3049. URL: <https://aclanthology.org/W15-3049/>.
- [25] Simon Kornblith et al. “Similarity of Neural Network Representations Revisited”. In: *Proceedings of the 36th International Conference on Machine Learning*. 2019.
- [26] Le Song et al. “Feature Selection via Dependence Maximization”. In: *Journal of Machine Learning Research* 13 (2012), pp. 1393–1434.
- [27] J. G. Wendel. “A Problem in Geometric Probability”. In: *Mathematica Scandinavica* 11 (1962), pp. 109–111.